A project report on

RESIDUAL GRAPH INDEXING: AUDITING AND REPAIRING INDEX-TIME INFORMATION LOSS IN COMMUNITY-BASED GRAPHRAG

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

JEYAVIKNAN S.R (23BCE1766)

ADITHYA D.S (23BCE1864)

AATREYEE DEV (23BCE1996)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

RESIDUAL GRAPH INDEXING: AUDITING AND REPAIRING INDEX-TIME INFORMATION LOSS IN COMMUNITY-BASED GRAPHRAG

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

JEYAVIKNAN S.R (23BCE1766)

ADITHYA D.S (23BCE1864)

AATREYEE DEV (23BCE1996)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled “Residual Graph Indexing: Auditing and Repairing Index-Time Information Loss in Community-Based GraphRAG” submitted by Jeyaviknan S.R (23BCE1766), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Karthika Veeramani.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "Residual Graph Indexing: Auditing and Repairing Index-Time Information Loss in Community-Based GraphRAG" is prepared and submitted by Jeyaviknan S.R (23BCE1766) to Vellore Institute of Technology, Chennai, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record carried out under my guidance. The project fulfils the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Karthika Veeramani

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,
B.Tech. Computer Science and Engineering

Name: Dr. Prassanna J

Date:

(Seal of the School of Computer Science and Engineering)

ABSTRACT

Graph-based retrieval-augmented generation (GraphRAG) answers questions about a whole document collection by reading summaries of graph communities instead of the graph itself. A large language model writes each summary from a summary input of at most 8,000 tokens, so anything that never reaches an input is lost before a question is asked, and no retrieval method that reads the summaries can recover it. This project measured that loss in Microsoft's GraphRAG 3.1.1 and built a method that repairs it within the same budget.

The loss happens at community boundaries, at the token budget, in the step that chooses which relationships an input may contain, when a large community is summarized from its children's summaries, and for entities that are never clustered. We separate relationships that are out of scope for their own community from relationships whose evidence is absent from every input. Running GraphRAG's own indexing code without a language model on all 20 novels of GraphRAG-Bench Novel, we found that at the default query level the summary inputs keep only 12.4% of 116,217 relationships in scope, and that at the root level 87.6% of relationships have no evidence in any input. The standard input builder admits only 4.6% to 16.3% of within-community relationships, whatever the budget.

We propose Residual Graph Indexing (RGI), which keeps GraphRAG's clustering, summaries and search, and adds an exact record of what the summaries leave out. It spends each input in two tiers, writes bridge records across communities, stores the rest and certifies that every relationship is accounted for. At the same budget and with no extra model calls, RGI raises the in-scope share at the default level from 12.4% to 34.6% without dropping anything GraphRAG kept. A tested toolkit, the GraphRAG X-Ray simulator and a pre-registered answer-level protocol complete the work.

Keywords: retrieval-augmented generation, knowledge graphs, GraphRAG, community detection, summarization, information loss

ACKNOWLEDGEMENT

I am deeply grateful to Dr. Karthika Veeramani, Assistant Professor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for her constant guidance, encouragement and understanding throughout this project. Above all, she taught me patience in my work. My association with her has gone beyond academics, and it has been a real opportunity to work with someone whose expertise lies in big data analytics and machine learning.

I would like to thank the leadership of the institute for providing an excellent working environment and for inspiring all of us during the course: Dr. G. Viswanathan, our Honorable Chancellor; Mr. Sankar Viswanathan, Dr. Sekar Viswanathan and Dr. G V Selvam, Vice Presidents; Dr. Sandhya Pentareddy, Executive Director; Ms. Kadhambari S. Viswanathan, Assistant Vice-President; Dr. V. S. Kanchana Bhaaskaran, Vice-Chancellor; Dr. T. Thyagarajan, Pro-Vice Chancellor, VIT Chennai; and Dr. P. K. Manoharan, Additional Registrar.

I also thank Dr. Viswanathan V, Dean, and Dr. Nithyanandam P, Dr. Suganya G and Dr. Sweetlin Hemalatha C, Associate Deans, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for the time and effort they spent sharing their knowledge and for helping us in every way they could.

I sincerely thank Dr. Prassanna J, Professor and Head of the Department, B.Tech. Computer Science and Engineering, and the Project Coordinators for their support and encouragement in taking up and completing this work.

My thanks go to my teammates, Adithya D.S and Aatreyee Dev, with whom this project was planned and built, and to all the faculty and staff of Vellore Institute of Technology, Chennai who helped me acquire the knowledge it needed. I thank my parents for their support, and my friends for encouraging me to take up and finish this task.

Place: Chennai

Date: **Jeyaviknan S.R**

CONTENTS

CONTENTS ..... **iii**

LIST OF FIGURES ..... **vi**

LIST OF TABLES ..... **vii**

LIST OF ACRONYMS ..... **viii**

Chapter 1

Introduction **1**

1.1 INTRODUCTION 1
1.2 OVERVIEW OF GRAPHRAG 2
1.3 CHALLENGES IN COMMUNITY-SUMMARY INDEXING 3
1.4 PROBLEM STATEMENT 4
1.5 OBJECTIVES 5
1.6 SCOPE OF THE PROJECT 5
1.7 CONTRIBUTIONS 6
1.8 ORGANIZATION OF THE REPORT 6

Chapter 2

Background and Literature Survey **8**

2.1 RETRIEVAL-AUGMENTED GENERATION 8
2.2 GRAPH-BASED RETRIEVAL-AUGMENTED GENERATION 9
2.2.1 GRAPHRAG AND LAZYGRAPHRAG 9
2.2.2 OTHER GRAPH-BASED SYSTEMS 9
2.3 COMMUNITY DETECTION AND ITS LIMITS 10
2.4 WORK ON COMMUNITY BOUNDARIES 11
2.5 COMPRESSION, GRANULARITY AND CONTEXT LENGTH 12
2.6 HOW GRAPH-BASED RAG HAS BEEN EVALUATED 13
2.7 KEYFRAMES AND RESIDUALS IN VIDEO CODING 13
2.8 PRE-REGISTRATION AND STATISTICAL TOOLS 14
2.9 RESEARCH GAP 14

Chapter 3

Requirements and Analysis **16**

3.1 REQUIREMENTS ANALYSIS 16
3.1.1 FUNCTIONAL REQUIREMENTS 16
3.1.2 NON-FUNCTIONAL REQUIREMENTS 17
3.2 HARDWARE AND SOFTWARE ENVIRONMENT 17
3.3 DATASET: GRAPHRAG-BENCH NOVEL 18
3.4 FEASIBILITY AND CONSTRAINTS 19

3.5 ANALYSIS OF THE GRAPHRAG 3.1.1 SOURCE 19
3.6 PRE-REGISTERED EVALUATION DESIGN 21

Chapter 4

System Design and Methodology 23
4.1 SYSTEM ARCHITECTURE 23
4.2 METHODOLOGY AND WORKFLOW 24
4.3 MODEL OF INDEX-TIME LOSS 25
4.3.1 NOTATION 25
4.3.2 STATUS OF A RELATIONSHIP 25
4.3.3 SCOPE AND EVIDENCE 26
4.3.4 THE HIERARCHY TRADE-OFF 27
4.4 THREE PROPOSITIONS 28
4.5 RESIDUAL GRAPH INDEXING 29
4.5.1 DESIGN CONSTRAINTS 29
4.5.2 LOSS AUDIT 30
4.5.3 TWO-TIER SUMMARY INPUT 30
4.5.4 BRIDGE RECORDS 31
4.5.5 RESIDUAL STORE AND TOKEN-EXACT SUBSTITUTION 31
4.5.6 COVERAGE CERTIFICATE 31
4.5.7 ALGORITHM 31
4.5.8 GUARANTEES 33
4.6 DESIGN OF THE MEASUREMENT TOOLKIT 33
4.7 GRAPHRAG X-RAY SIMULATOR DESIGN 34

Chapter 5

Implementation 36
5.1 REPOSITORY AND DEVELOPMENT ENVIRONMENT 36
5.2 MEASUREMENT TOOLKIT 36
5.2.1 SCHEMA AND INPUT HANDLING 37
5.2.2 LOSS COMPUTATION 37
5.2.3 LINKING, RECOVERABILITY AND ATTRIBUTION 38
5.2.4 TOKEN-EXACT INJECTION 38
5.2.5 EXPERIMENT SCRIPTS 39
5.3 CENSUS PIPELINE 40
5.3.1 INDEXING WITH GRAPHRAG'S OWN CODE 40
5.3.2 EXACT ADMISSION 41
5.3.3 STATUS, EVIDENCE AND RGI INPUTS 42
5.4 LINKING GOLD EVIDENCE 43
5.5 SELECTION IN THE STANDARD PIPELINE 43
5.6 ANALYSIS AND REPRODUCIBILITY 43

5.7 GRAPHRAG X-RAY SIMULATOR....44
5.7.1 BUILDING THE SIMULATED INDEX....44
5.7.2 INTERFACE....45
5.7.3 TYPED QUESTIONS....47
5.8 TESTING....48

Chapter 6

Results and Discussion....50
6.1 EXPERIMENTAL SETUP....50
6.2 HOW MUCH EACH LEVEL LOSES....51
6.3 SCOPE AND EVIDENCE....53
6.4 SELECTION IN THE STANDARD PIPELINE....53
6.5 WHAT RGI RECOVERS AT THE SAME BUDGET....54
6.6 IS THE LOST CONTENT WHAT QUESTIONS NEED?....56
6.7 SENSITIVITY AND ABLATIONS....58
6.8 CONTROLLED SIMULATION....59
6.9 DISCUSSION....61
6.9.1 SCOPE AND EVIDENCE ARE DIFFERENT LOSSES....61
6.9.2 WHAT RGI DOES AND DOES NOT DO....61
6.9.3 ALLOCATION DEPENDS ON THE LEVEL....61
6.9.4 RECONCILING WITH EARLIER EVIDENCE....61
6.10 THREATS TO VALIDITY....62

Chapter 7

Conclusion and Future Work....63
7.1 CONCLUSION....63
7.2 OBJECTIVES REVISITED....63
7.3 WHAT THE PROJECT SHOWED....64
7.4 LIMITATIONS....65
7.5 FUTURE WORK....65
7.6 BENEFITS OF THE WORK....66

REFERENCES....67

APPENDICES....71
APPENDIX 1: PRE-REGISTERED KILL CRITERIA....71
APPENDIX 2: RGI SUMMARY-INPUT CONSTRUCTION....73
APPENDIX 3: CENSUS OUTPUT FILES....75
APPENDIX 4: GRAPHRAG X-RAY USER GUIDE....76
APPENDIX 5: REPRODUCTION GUIDE AND PROJECT TIMELINE....78

LIST OF FIGURES

Figure 1.1: GraphRAG indexing pipeline and global search....2
Figure 3.1: Question types in GraphRAG-Bench Novel....18
Figure 4.1: System architecture of the project....23
Figure 4.2: Project workflow....24
Figure 4.3: Where content leaves the path to an answer (red) and what RGI adds (green)....26
Figure 4.4: Boundary loss at a fine level and compression at a coarse level....28
Figure 4.5: How GraphRAG and RGI fill the same summary input....30
Figure 4.6: Construction of one RGI summary input....32
Figure 4.7: Structure of the measurement toolkit....34
Figure 4.8: How the GraphRAG X-Ray simulator is organized....35
Figure 5.1: Stages of the census pipeline for one novel....40
Figure 5.2: GraphRAG X-Ray: original GraphRAG at the leaf level....45
Figure 5.3: GraphRAG X-Ray: the same question under RGI....46
Figure 5.4: GraphRAG X-Ray at the root level under RGI....46
Figure 5.5: A typed question resolved to a three-relationship chain....48
Figure 6.1: Size of each novel's graph after pruning....51
Figure 6.2: Where every relationship ends up at each query level....52
Figure 6.3: Measured share of within-community relationships in the standard summary input, set against the bound....54
Figure 6.4: In-scope share at level 2 for each novel, GraphRAG and RGI....55
Figure 6.5: In-scope share against the summary input budget....56
Figure 6.6: Share of linked questions with a gold relationship out of scope....57
Figure 6.7: Simulated share of evidence retrieved at each level....59
Figure A4.1: Question menu with featured questions and a field for typed questions....77

LIST OF TABLES

Table 2.1: Approaches to the space between communities....12
Table 2.2: Gaps in prior work and how this project addresses them....15
Table 3.1: Functional requirements....16
Table 3.2: Non-functional requirements....17
Table 3.3: Software environment....17
Table 3.4: Differences between the documentation and GraphRAG 3.1.1....20
Table 3.5: Kill criteria committed before any data existed....21
Table 4.1: Status of a relationship r = (a, b) at level ℓ....25
Table 4.2: Evidence tiers....27
Table 5.1: Main folders of the project repository....36
Table 5.2: Modules of the measurement toolkit....37
Table 5.3: Experiment scripts....39
Table 5.4: Commands that regenerate the results....44
Table 5.5: Test suite of the measurement toolkit....48
Table 6.1: Census corpus and index....50
Table 6.2: Share of relationships (%) by status at B = 8,000....51
Table 6.3: Summary inputs at B = 8,000, summed over all novels....52
Table 6.4: Strongest form in which evidence reaches a selected input (%), B = 8,000....53
Table 6.5: How many within-community relationships reach the standard pipeline's summary input....53
Table 6.6: Linking gold triples to the census graphs....56
Table 6.7: Share of questions (%) with at least one gold relationship out of scope....57
Table 6.8: Questions with a gold relationship out of scope at level 2, by type (%)....57
Table 6.9: Sensitivity to the text-unit size (in-scope share, %, B = 8,000)....58
Table 6.10: RGI ablations at B = 8,000 (% of relationships)....58
Table 6.11: Controlled simulation results (synthetic data, 48 questions, %)....59
Table 6.12: Simulation results under each configuration (%)....60
Table 6.13: Threats to validity and how they were handled....62
Table 7.1: Objectives and outcomes....64
Table A1.1: Strength markers committed with the gates....72
Table A1.2: Frozen linking parameters....72
Table A3.1: Files written by the census....75
Table A3.2: Main columns of the level census....75
Table A4.1: Controls of GraphRAG X-Ray....76
Table A5.1: Project timeline....78

LIST OF ACRONYMS

Acronym	Expansion
API	Application Programming Interface
AVC	Advanced Video Coding (H.264/AVC)
CI	Confidence Interval
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
E0 to E9	Experiment scripts of the measurement toolkit
F1 to F10	Functional requirement identifiers
G1 to G4	Pre-registered kill criteria (gates)
GraphRAG	Graph-based Retrieval-Augmented Generation (Microsoft)
HTML	HyperText Markup Language
ID	Identifier
JSON	JavaScript Object Notation
LCC	Largest Connected Component
LLM	Large Language Model
MIT	Massachusetts Institute of Technology (licence name)
NLI	Natural Language Inference
NLTK	Natural Language Toolkit
OMD-GraphRAG	Ontology-guided, Multi-dimensional, Dual-channel GraphRAG
PMI	Pointwise Mutual Information
RAG	Retrieval-Augmented Generation
RAPTOR	Recursive Abstractive Processing for Tree-Organized Retrieval
RECOMP	Retrieve, Compress, Prepend
RGI	Residual Graph Indexing

LIST OF SYMBOLS

Symbol	Meaning
$G = (V, E)$	Extracted knowledge graph with entities V and relationships E
$r = (a, b)$	Relationship between entities a and b
U, $U(r)$	Set of text units; the text units that support relationship r
C, c	Set of communities; one community
$\lambda(c)$	Level of community c in the hierarchy (0 is the coarsest)
ℓ	Query level of global search
$\sigma\ell(v)$	Deepest community at or above level ℓ that contains entity v

$S\ell$	Communities whose summaries global search reads at level ℓ
I(c)	Summary input of community c
B	Token budget of a summary input (8,000 by default)
T(x)	Number of tokens in text x under the o200k_base encoding
$L\ell$	Relationships out of scope at level ℓ
R(G, ℓ), R(RGI, ℓ)	Relationships in scope and represented under GraphRAG and under RGI
|c|	Number of entities in community c
$\bar{d}$	Mean number of internal relationships per entity of a community
f	RGI full-item cutoff, as a share of B (0.8 by default)
b	RGI bridge share of B (0.12 by default)
m	Minimum number of supporting text units for a bridge record (2 by default)
E_in, E_bd	In-scope and boundary relationships of a community
P, k_G	GraphRAG's items for a community; number of items GraphRAG admits
X, rep(X)	Input under construction; relationships represented in X
Y(q)	Gold relationships of question q
K	Number of summaries read per question in the simulation
κ	Cohen's kappa, a measure of agreement between two annotators

Chapter 1

Introduction

1.1 INTRODUCTION

Large language models answer questions fluently, but they answer from what they absorbed during training. When the question is about a private report, a recent policy document or a novel that the model never read closely, its answer can sound confident and still be wrong. Retrieval-augmented generation (RAG) addresses this by fetching relevant text from a document collection at question time and placing it in the model's context, so that the answer is grounded in the collection rather than in the model's memory (Lewis et al., 2020; Gao et al., 2023).

The common form of RAG splits documents into chunks, embeds each chunk as a vector, and retrieves the few chunks closest to the question. This works well when the answer sits in one or two passages. It works poorly when the question is about the collection as a whole. A question such as "What are the main conflicts in this novel?" or "Which themes run through these reports?" has no single passage that answers it. The evidence is spread across hundreds of pages, and the closest chunks to the question are usually not the ones that matter.

GraphRAG, introduced by Microsoft Research, was designed for these corpus-level questions (Edge et al., 2024a). Instead of retrieving raw chunks, it builds an index in advance. A language model reads the documents and extracts the entities they mention and the relationships between those entities. The resulting knowledge graph is divided into a hierarchy of communities, groups of entities that are more connected to each other than to the rest of the graph, using the Leiden algorithm (Traag et al., 2019). The language model then writes one summary per community. When a user asks a broad question, GraphRAG's global search reads the summaries at one level of the hierarchy, asks the model for a partial answer from each batch, and combines the partial answers into one response.

The appeal of this design is that the summaries give the model a compact picture of the whole corpus. What is easy to miss is how much then rests on those summaries. Global search does not read the graph, and it does not read the source text either; it

reads only the summaries, each of which was written from a token-limited selection of its community's content. If the summarizer was never shown some piece of content, a summary can contain it only through what the model happened to know already. This loss takes place while the index is being built, long before anyone asks a question.

Our project is about that loss. We wanted to know where GraphRAG drops content between the documents and the summaries, and how much it drops on a real benchmark. We also wanted to know whether the index could be changed so that the dropped content is kept, or at least recorded, at no extra cost; the method we developed for this is called Residual Graph Indexing (RGI).

1.2 OVERVIEW OF GRAPHRAG

The parts of GraphRAG that matter for this project are drawn in Figure 1.1. Indexing, in the upper half of the figure, is done once and offline, while global search in the lower half runs again for every question.

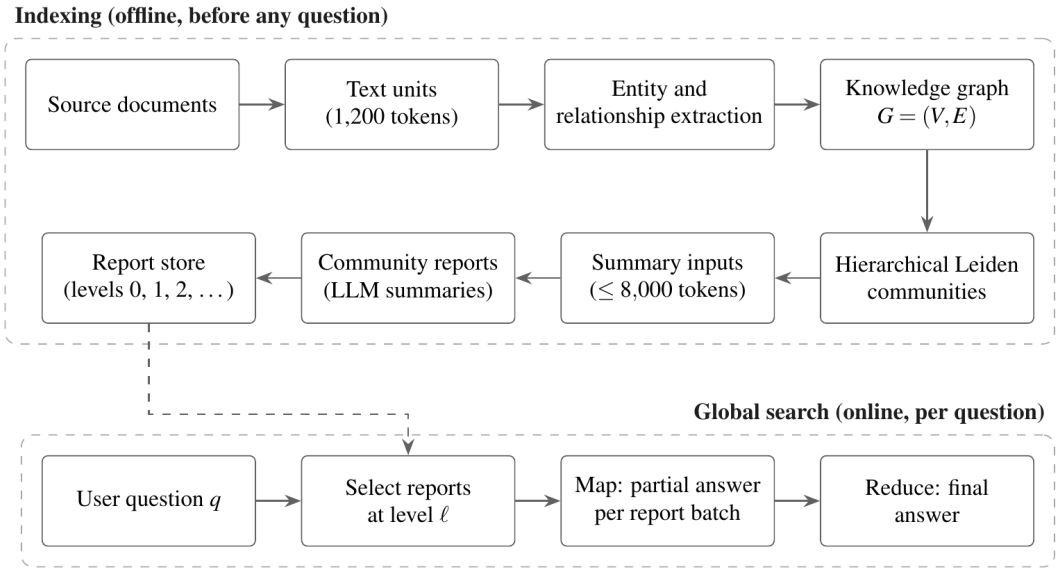


Figure 1.1: GraphRAG indexing pipeline and global search

Indexing starts by cutting each document into text units of 1,200 tokens, with 100 tokens of overlap between neighbouring units; tokens are counted with the `o200k_base` encoding. From these units GraphRAG can build the graph in two ways. In the standard pipeline every text unit goes to a language model, and the model sends back entities and relationships, each with a short description. The fast pipeline does not use a model for this step. It pulls noun phrases out of the text with a rule-based tagger,

connects phrases that occur in the same text unit and weights each connection by pointwise mutual information (PMI). After either pipeline, weak nodes and edges are pruned and the graph goes to hierarchical Leiden clustering, which keeps splitting a community for as long as it has more than ten entities. What comes out is a tree of communities in which level 0 holds the coarsest partition and every deeper level holds smaller, more tightly focused groups.

GraphRAG then builds a summary input for every community. With the standard pipeline that input lists the community's entities and relationships along with their descriptions, whereas with the fast pipeline it consists of the text units where the community's relationships appear. Items are added in priority order until the input reaches the budget, 8,000 tokens by default (`max_input_length`), and a language model writes the community report from whatever made it in. Communities whose input is too large are handled in one of two ways: a leaf community is simply cut at the budget, while a parent community is written from the reports of its children instead of its own content.

At query time, global search takes a level as a parameter, and unless told otherwise the command-line tool uses level 2. Every entity is read through its deepest community at or above that level, and the reports of those communities are batched, mapped to partial answers and reduced to a final answer. GraphRAG-Bench, a recent benchmark for graph-based RAG, measured 331,375 tokens per global-search query on its Novel corpus (Xiang et al., 2026), which gives a sense of how much text the reports carry into each answer.

1.3 CHALLENGES IN COMMUNITY-SUMMARY INDEXING

Three properties of this design make index-time loss both likely and hard to notice.

Communities partition the graph, and that is the first property. Each entity sits in exactly one community at a given level, which means a relationship whose two entities ended up in different communities is owned by neither: it lies outside both scopes, and in the standard pipeline it is missing from both summary inputs. Real corpora seldom split cleanly. In a novel, for example, the characters who link two

storylines are precisely the ones whose relationships cross from one community into another.

The fixed budget is the second property. A large community can easily hold more than 8,000 tokens of content, and GraphRAG, after sorting that content by how connected it is, stops at the first item that does not fit, so the details that disappear are the specific, weakly connected ones. Moving to a coarser level of the hierarchy reduces the number of boundaries, since communities merge, but more content then has to pass through the same budget, and a change of level ends up trading boundary loss for budget loss.

Third, nobody can see the loss. GraphRAG does not record what a summary input left out, and evaluations that score answer quality in aggregate miss element-level gaps as well, since a broad answer can read well while leaving out a particular fact. When GraphRAG was first evaluated, global search showed no significant difference from a summarization baseline that does not partition the graph at all (Edge et al., 2024a). We take that as useful evidence. It was, however, a measure of aggregate quality, and it says little about how much specific content the index drops.

Improving the retriever would not help with any of this. Retrieval chooses which summaries are read but has no say in what they contain, so a relationship that never entered a summary input stays hidden from every retrieval policy that works from summaries. Any repair therefore has to be made in the index itself.

1.4 PROBLEM STATEMENT

Community-summary GraphRAG discards part of the extracted knowledge graph while it builds its index, through community boundaries, a fixed input budget, the rules that assemble summary inputs and the hierarchical substitution of child summaries. The discarded content is not recorded, its size on real data is unknown, and no retrieval policy that reads the summaries can recover it. The problem addressed by this project is to measure that loss exactly on a public benchmark using GraphRAG's own code, and to design an indexing method that keeps the summaries, clustering and search of GraphRAG unchanged, uses the same token budget and no additional language model calls, never drops content that GraphRAG kept, and accounts for every relationship in the graph.

1.5 OBJECTIVES

The project set out to meet the following objectives.

1. Build a precise model of index-time loss in community-summary GraphRAG, separating content that is merely out of a community's scope from content whose evidence never reaches any summary input.
2. Develop a tested measurement toolkit for GraphRAG's index files, and commit the thresholds that would stop the study to version control before collecting any data.
3. Measure the loss exactly on all twenty novels of the GraphRAG-Bench Novel corpus by running the shipped GraphRAG 3.1.1 code, at every query level and at several budgets.
4. Design Residual Graph Indexing as an addition to GraphRAG that comes with stated guarantees on budget, monotonicity, cost and coverage, and check that those guarantees hold on every run.
5. Find out how much of the lost content the benchmark questions actually need, and see how the results shift when the text-unit size or RGI's parameters change.
6. Build an interactive simulator, GraphRAG X-Ray, that shows the mechanism and its effect on retrieval and on judged answers in a controlled setting.

1.6 SCOPE OF THE PROJECT

The project works with Microsoft GraphRAG version 3.1.1 and with the Novel corpus of GraphRAG-Bench, which has twenty public-domain novels and 2,010 questions that come with gold evidence. Every measurement on real data was made with GraphRAG's fast indexing pipeline, which builds the graph and its communities without calling a language model. We chose it because it let us run an exact census on the budget we had, and because anyone can repeat the measurements on an ordinary laptop. The input builder of the standard pipeline is analysed on its own, by running its code on the same graphs.

A few questions are outside what this report covers. No index was built with a language model, so there is no measurement here of answer accuracy on real data. When we tried to start that run, the application programming interface (API) key the team had failed validation, and we decided to finish every part of the study that did not

depend on it. The answer-level test is fully specified, its thresholds were frozen in advance, and its tooling is implemented and tested, so running it is the main piece of future work. Effects on answers in the simulator come from a model we wrote ourselves, and we report them that way. We did not evaluate the Medical corpus of GraphRAG-Bench or other graph-based systems such as LightRAG.

1.7 CONTRIBUTIONS

The project makes the following contributions.

- A loss model for community-summary GraphRAG with three propositions. Content whose evidence is absent from every summary input is unreachable for any retrieval policy that reads summaries; boundary loss can only grow toward the leaves of the hierarchy; and GraphRAG's standard input builder admits at most $2/\bar{d}$ of a community's internal relationships, where $\bar{d}$ is their mean internal degree.
- An exact census of index-time loss on 20 novels and 116,217 relationships, produced by GraphRAG 3.1.1's own code with no language model calls, and checked against GraphRAG's own context builder on 139 communities.
- The finding that GraphRAG's standard input builder passes only 4.6% to 16.3% of within-community relationships into summary inputs, independent of the budget.
- Residual Graph Indexing, with four guarantees: no input exceeds its budget, no relationship that GraphRAG kept in scope is dropped, the number of language model calls is unchanged, and every relationship is certified as summarized, bridged or stored. RGI raises the in-scope share at the default level from 12.4% to 34.6%.
- A measurement toolkit of nine Python modules and ten experiment scripts with 166 passing tests, kill criteria committed before any data existed, and the GraphRAG X-Ray simulator with support for typed questions.

1.8 ORGANIZATION OF THE REPORT

The remaining chapters are arranged as follows. Chapter 2 reviews retrieval-augmented generation, graph-based retrieval, community detection and recent work on

community boundaries, and points out the gap this project addresses. Chapter 3 covers the requirements, the dataset, the software environment and the pre-registered evaluation design. The system architecture, the loss model and the design of Residual Graph Indexing are in Chapter 4, and Chapter 5 explains how the measurement toolkit, the census pipeline, the analysis scripts and the GraphRAG X-Ray simulator were implemented and tested. Results on GraphRAG-Bench and in simulation, together with a discussion of their limits, make up Chapter 6. Chapter 7 closes the report with conclusions and future work, and the references and appendices follow it.

Chapter 2

Background and Literature Survey

The work this project builds on is reviewed here. We begin with retrieval-augmented generation in general and then turn to graph-based retrieval and GraphRAG itself, to the community detection methods GraphRAG relies on, and to recent systems that try to handle what falls between communities. Later sections deal with context compression, with how graph-based RAG has been evaluated, with the video-coding idea behind our design and with the statistics used in the evaluation plan, and the chapter closes with the gap this project is meant to fill.

2.1 RETRIEVAL-AUGMENTED GENERATION

Retrieval-augmented generation was introduced by Lewis et al. (2020), who combined two kinds of memory in one model: knowledge stored in the weights of a pretrained sequence-to-sequence model, and knowledge stored outside it in a dense vector index of documents. For every input, a neural retriever picks documents from the index, and the generator conditions its output on the input together with what was retrieved. Compared with the generator on its own, this gave more specific and more factual answers on knowledge-intensive tasks, open-domain question answering among them.

RAG has since become the usual way of connecting language models to private or frequently changing data. In their survey, Gao et al. (2023) trace three stages in its development. The first, naive RAG, indexes chunks, retrieves the most similar ones and hands them to the model; advanced RAG adds steps before and after retrieval, for instance query rewriting and reranking; and modular RAG treats retrieval, generation and the steps around them as components that can be rearranged to suit a task.

Chunk-based RAG has a structural limit that matters for this project. It retrieves passages by similarity to the question, so it can only return text that resembles the question. Questions about a collection as a whole, for example about its recurring themes or the main relationships among its people, are not similar to any single passage.

Increasing the number of retrieved chunks helps only a little, because the model then receives a long and unordered context in which the relevant parts are hard to use.

2.2 GRAPH-BASED RETRIEVAL-AUGMENTED GENERATION

Graph-based RAG adds an explicit structure between the documents and the retriever. Peng et al. (2024) organize the area into three steps: building a graph index, retrieving from the graph, and generating with the retrieved graph content. Systems differ in what the graph contains, how it is searched and how its content is turned into text for the model.

2.2.1 GRAPHRAG AND LAZYGRAPHRAG

GraphRAG (Edge et al., 2024a) builds an entity and relationship graph with a language model, partitions it into a hierarchy of communities with Leiden, and writes a report for every community. It was designed for query-focused summarization over a whole corpus, which the authors call global sensemaking. Global search maps the question over community reports at a chosen level and reduces the partial answers. The authors compared this with naive chunk retrieval using a language model as judge and reported more comprehensive and more diverse answers. They also compared global search at several hierarchy levels with a summarization baseline that applies the same map and reduce steps directly to the source text, with no graph and no partition. That comparison found no significant difference in aggregate answer quality. We return to this finding in Chapter 6, because it bears directly on how large an effect index-time loss can have.

The open-source implementation has grown considerably since the paper. Version 3.1.1, used throughout this project, offers the standard pipeline, a fast pipeline that builds the graph from noun phrases without a language model, local search for entity-specific questions, and global search for corpus-level ones (Microsoft, 2026). LazyGraphRAG (Edge et al., 2024b) takes a different route and postpones summarization until query time, which removes most of the upfront indexing cost.

2.2.2 OTHER GRAPH-BASED SYSTEMS

Several systems keep a graph but drop the community layer. LightRAG (Guo et al., 2024) builds an entity and relationship index and retrieves at two levels: specific entities and their neighbours for detailed questions, and broader relational keys for

abstract ones. HippoRAG (Gutiérrez et al., 2024) takes its inspiration from the role of the hippocampus in human memory. It extracts a schemaless knowledge graph and runs personalized PageRank from the entities in the question, which suits multi-hop questions. G-Retriever (He et al., 2024) targets textual graphs and selects a connected subgraph for each question by solving a prize-collecting Steiner tree problem.

Other systems keep a hierarchy but build it differently. RAPTOR, short for recursive abstractive processing for tree-organized retrieval (Sarthi et al., 2024), recursively clusters and summarizes text chunks into a tree and retrieves from any level of it, so that a question can be matched to detailed passages or to high-level summaries. ArchRAG (S. Wang et al., 2026) detects communities from both links and node attributes and organizes them into a hierarchical index for fast retrieval across levels.

2.3 COMMUNITY DETECTION AND ITS LIMITS

Because GraphRAG's index is built on community detection, whatever properties the clustering method has end up in the index as well. The Louvain method (Blondel et al., 2008) optimizes modularity greedily, alternating between moving single nodes from one community to another and merging communities into super-nodes, and its speed on large networks made it popular. Traag et al. (2019) then showed that Louvain can return communities that are poorly connected, sometimes even disconnected. Their Leiden algorithm adds a refinement phase and guarantees that communities are well connected. GraphRAG uses a hierarchical Leiden implementation from the graspologic library, which succeeded GraSPy (Chung et al., 2019).

Modularity has well-known limits of its own. Fortunato and Barthélemy (2007) proved that optimizing it can miss communities below a certain scale, one that depends on the size of the whole network; this is known as the resolution limit. According to Good et al. (2010), real networks also admit a very large number of partitions whose modularity is almost the same but whose structure differs, which makes the partition an algorithm returns partly arbitrary. Fortunato (2010) reviews these issues and others at length. Some nodes are hard to place by their nature: Guimerà and Amaral (2005) describe connector nodes with links spread over several modules and measure this with the participation coefficient.

These results matter here for a simple reason. If the partition is partly arbitrary, then the boundaries it draws are partly arbitrary too, and so is the set of relationships that fall on them. Hossain and Sarıyüce (2026) act on this directly for GraphRAG. They prove that on sparse knowledge graphs modularity admits exponentially many near-optimal partitions, replace Leiden with a deterministic k-core decomposition, and add a sampling strategy that respects a token budget. Their work changes the clustering. This project leaves the clustering alone and instead measures and repairs what any partition leaves out.

2.4 WORK ON COMMUNITY BOUNDARIES

The gaps between communities have drawn attention from several recent systems. Most of them address the problem at query time.

HiRAG (Huang et al., 2025) builds a layered knowledge hierarchy and, at query time, retrieves knowledge that connects local entity descriptions with global community summaries. LeanRAG (Zhang et al., 2026) observes that the summaries in hierarchical graph RAG tend to become unconnected "semantic islands". It aggregates entities layer by layer, creates explicit relations between the aggregated summary nodes, and retrieves along paths through their lowest common ancestor. Deep GraphRAG (Li et al., 2026) retrieves from global to local in stages: it filters across communities, refines at the community level, and then searches entities inside the chosen communities, and it trains a small model with reinforcement learning to integrate the results.

The closest work to this project is OMD-GraphRAG, whose name stands for ontology-guided extraction, multi-dimensional clustering and dual-channel fusion (J. Wang et al., 2026). Among other changes to the GraphRAG pipeline, it adds a completion step for community boundaries. When an entity outside a community has a large enough share of its edges pointing into the community, controlled by a threshold that the authors set to 0.5, its attributes and relations are projected into that community's summary representation. The system is evaluated on MultiHop-RAG.

Table 2.1 compares these systems with the approach taken in this project. The distinguishing features of RGI are that it starts from an exact account of what the summary inputs leave out, works inside the existing token budget, adds no language

model calls, and checks at build time that every relationship is accounted for. To our knowledge none of the systems above reports an element-level account of what the summaries omit.

Table 2.1: Approaches to the space between communities

System	Acts at	Mechanism
GraphRAG (Edge et al., 2024a)	Index	One summary per community; relationships across communities fall outside every scope
HiRAG (Huang et al., 2025)	Index and query	Layered knowledge hierarchy; retrieval of knowledge connecting local and global levels
LeanRAG (Zhang et al., 2026)	Index and query	Explicit relations among aggregated summary nodes; lowest-common-ancestor path retrieval
Deep GraphRAG (Li et al., 2026)	Query	Inter-community filtering followed by intra-community search
OMD-GraphRAG (J. Wang et al., 2026)	Index	Threshold-based projection of boundary entities into summaries
Core hierarchies (Hossain and Sarıyüce, 2026)	Index	k-core hierarchy replaces Leiden; budget-aware sampling
RGI (this project)	Index and query	Exact loss audit; bridge records and compact facts inside the same budget; residual store; coverage certificate

2.5 COMPRESSION, GRANULARITY AND CONTEXT LENGTH

Any RAG system gives up some detail to keep its context bounded, and several lines of work look at that trade. RECOMP, short for retrieve, compress and prepend (Xu et al., 2024), turns retrieved documents into short summaries before the model sees them, using extractive or abstractive compressors, and it may decide to add nothing at all when the documents would not help. Chen et al. (2024) study what the unit of retrieval should be and find that propositions, short statements of a single fact, can work better than passages or sentences. We took their result as support for using one-line compact facts as a cheap unit that still preserves a relationship.

Liu et al. (2024) found that language models make uneven use of long contexts. Performance was best when the relevant information came at the start or the end of the context and fell when it was in the middle. That finding led to one of our design rules. When content is added on top of an existing context, a change in the answers might be caused by the extra length instead of the content, and so RGI never adds tokens: whatever it adds replaces other content, keeping the token count the same, and the planned answer-level test compares two injections of exactly the same length.

2.6 HOW GRAPH-BASED RAG HAS BEEN EVALUATED

Whether graph structure helps is still an open question. In a systematic comparison of RAG and GraphRAG on text benchmarks, Han et al. (2025) found that neither wins overall, since each does better on certain kinds of question. Zhou et al. (2025) put many graph-based RAG methods into a single framework and compare their components on question answering. Xiang et al. (2026) asked directly when graphs help, and built GraphRAG-Bench to find out. The benchmark has two corpora: a Novel corpus of pre-twentieth-century novels from Project Gutenberg, picked from lesser-known works so that they overlap less with model training data, and a Medical corpus built from clinical guidelines. Its questions are of four types, in increasing difficulty (fact retrieval, complex reasoning, contextual summarization and creative generation), and it evaluates every stage of the pipeline, from graph construction through to the final answer. For multi-hop questions over news articles there is MultiHop-RAG (Tang and Yang, 2024).

We drew two lessons from this literature for our evaluation. Aggregate answer quality can conceal large differences in which specific facts a system is able to reach. And global search is expensive, at 331,375 tokens per query on the Novel corpus of GraphRAG-Bench (Xiang et al., 2026), so a repair that put more tokens into every query would be hard to defend. That cost is one more reason RGI keeps the budget fixed.

2.7 KEYFRAMES AND RESIDUALS IN VIDEO CODING

We took both the name and the main idea of RGI from video compression. The H.264/AVC (Advanced Video Coding) standard (Wiegand et al., 2003) does not store every frame in full. Intra-coded frames, usually called keyframes, are coded independently, while most of the remaining frames are predicted from reference frames, and for those the encoder keeps only the residual, meaning the difference between the predicted frame and the real one. At playback the decoder adds this residual back onto the prediction. Since predictions are usually good, the residual is small, and since it holds exactly what the prediction got wrong, nothing important is lost.

In our design the community summaries take the place of keyframes, being compact and carrying most of what a question needs, and the residual is the set of relationships they leave out. GraphRAG's own index tables already record which

relationships belong to each community, so we can compute the residual exactly, as a set difference, instead of asking a language model to guess at it.

2.8 PRE-REGISTRATION AND STATISTICAL TOOLS

In a measurement study, many choices can be made after the data have been seen, for example which threshold should count as a large effect. Nosek et al. (2018) argue that an analysis plan written down before the data are seen keeps tests of a hypothesis apart from the exploration that suggested it. We followed this advice and committed our kill thresholds to version control before any data existed (Section 3.6).

The planned analysis uses standard tools. Confidence intervals are computed by the bootstrap (Efron, 1979), resampling questions or novels with replacement. Paired yes or no outcomes, such as whether the same question is answered correctly under two conditions, are compared with McNemar's test (McNemar, 1947). Agreement between two human annotators is measured with Cohen's kappa (Cohen, 1960). The planned recoverability check uses a cross-encoder model for natural language inference (NLI) based on DeBERTa (He et al., 2021), but only after the two annotators have shown that they agree with each other.

2.9 RESEARCH GAP

The literature leaves a specific gap. Existing work changes how communities are formed (Hossain and Sarıyüce, 2026; S. Wang et al., 2026) or how the space between them is searched at query time (Huang et al., 2025; Zhang et al., 2026; Li et al., 2026). The completion step of OMD-GraphRAG adds boundary information to summaries but selects it by a fixed threshold, without a budget and without an account of what remains lost (J. Wang et al., 2026). Evaluations measure answer quality in aggregate (Edge et al., 2024a; Han et al., 2025; Xiang et al., 2026), which cannot show which specific relationships a summary-based index can no longer reach.

As far as we know, no earlier study has measured exactly how much of the extracted graph GraphRAG's summary inputs leave out, broken down by level and by mechanism, with GraphRAG's own code on a public benchmark. We also found no repair that keeps the summaries and the budget unchanged, adds no model calls and

proves that nothing is dropped without a record. Table 2.2 shows how this project deals with each part of that gap.

Table 2.2: Gaps in prior work and how this project addresses them

Gap in prior work	How this project addresses it
The amount of index-time loss in GraphRAG has not been measured	Exact census on 20 novels with GraphRAG 3.1.1's own code (Chapter 6)
Scope and evidence are not distinguished	A loss model with evidence tiers and three propositions (Chapter 4)
The standard input builder's selection behaviour is unquantified	Measurement by running the builder unchanged (Section 6.4)
Boundary repairs add content without a budget or an account	RGI: same budget, no extra calls, coverage certificate (Section 4.5)
Answer-level claims are made without pre-registered thresholds	Kill criteria committed before data; tested toolkit (Section 3.6)

Chapter 3

Requirements and Analysis

Here we set out what the system had to do and the conditions it had to work under. The chapter also describes the dataset and the software environment, and it records two pieces of analysis done before any measurement: a reading of GraphRAG's source code, and an evaluation plan whose thresholds were fixed in advance.

3.1 REQUIREMENTS ANALYSIS

Three parts of the project share data and code. A measurement layer reads GraphRAG's index and computes the loss, a repair layer builds RGI's summary inputs, and a demonstration layer lets a user explore the mechanism interactively. All three are covered by the requirements that follow.

3.1.1 FUNCTIONAL REQUIREMENTS

Table 3.1 lists the functional requirements. Each one is traced to the part of the report where it is met.

Table 3.1: Functional requirements

ID	Requirement	Where it is met
F1	Build the knowledge graph, communities and summary inputs for every novel using GraphRAG 3.1.1's own code	Section 5.3
F2	Assign every relationship a status and an evidence tier at each query level and budget	Section 4.3
F3	Reproduce GraphRAG's admission rule exactly and verify it against GraphRAG's own function	Section 5.3
F4	Build RGI summary inputs within the same budget, with bridge records, compact facts and a residual store	Section 4.5
F5	Check RGI's guarantees on every run and stop if any fails	Section 5.3
F6	Link the benchmark's gold evidence to graph relationships with parameters fixed in advance	Section 5.4
F7	Measure which relationships GraphRAG's standard input builder passes into summary inputs	Section 5.5
F8	Produce every figure, table and reported number from the result files, without manual copying	Section 5.6
F9	Provide experiment scripts E0 to E9 and the four pre-registered gates for the answer-level test	Section 5.2
F10	Provide an interactive simulator that compares GraphRAG and RGI by level and accepts typed questions	Section 5.7

3.1.2 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements came mostly from the nature of the study. A measurement of loss is only useful if it can be trusted, so exactness and reproducibility ranked above speed and convenience. Table 3.2 lists them.

Table 3.2: Non-functional requirements

Requirement	What it means in this project
Exactness	Every count is a census of all relationships, not a sample. Where GraphRAG's behaviour is replayed, the replay gives the same result as GraphRAG's own code
Fidelity	Measurements use GraphRAG's shipped defaults and call its functions wherever possible; any substitution is disclosed
No model in the measurement path	The census and the RGI construction make no language model calls, so their results depend neither on a model's randomness nor on a paid service
Budget neutrality	RGI never uses more tokens than GraphRAG for a summary input or a query context
Reproducibility	One command per experiment regenerates every result from the public dataset; seeds are fixed and GraphRAG's clustering seed stays at its default
Testability	The toolkit is tested on a synthetic index whose correct answers are known by construction, and a failing test stops the work
Performance	A census of all twenty novels, with every configuration, finishes in minutes on a laptop
Portability	The simulator runs in a browser from local files, with no server and no external libraries

3.2 HARDWARE AND SOFTWARE ENVIRONMENT

All work was done on an Apple M2 laptop. Table 3.3 lists the main software and the role of each component. The Python environment is pinned in the project's lock file, which records 153 package versions.

Table 3.3: Software environment

Component	Version	Role in the project
Python	3.11.5	Toolkit, census, analysis
graphrag (with graphrag-chunking, graphrag-llm and other sub-packages)	3.1.1	System under study; chunking, edge building, pruning, clustering and input builders
graspologic-native	1.2.5	Hierarchical Leiden clustering called by GraphRAG
tiktoken	0.13.0	Token counting with the `o200k_base` encoding

Component	Version	Role in the project
pandas / numpy / pyarrow	3.0.5 / 2.4.6 / 25.0.1	Tables, arrays and GraphRAG's parquet outputs
RapidFuzz	3.14.5	Fuzzy matching of gold evidence to entity titles
scipy / statsmodels	1.17.1 / 0.14.6	Bootstrap intervals, McNemar's test, Cohen's kappa
matplotlib	3.11.1	Charts
pytest	9.1.1	Test suite
litellm / transformers / torch	1.92.0 / 5.15.0 / 2.13.0	Model access and NLI for the planned answer-level test
Node.js	23.9.0	Headless runs of the simulator
HTML, CSS and JavaScript (no frameworks)	n/a	GraphRAG X-Ray simulator
TeX Live	2026	Diagrams and the research paper

3.3 DATASET: GRAPHRAG-BENCH NOVEL

The project uses the Novel corpus of GraphRAG-Bench (Xiang et al., 2026), released under the MIT licence. It contains 20 novels with 4,819,610 characters in total, which come to 1,217,195 tokens under the `o200k_base` encoding. The novels range from about 93,000 to about 450,000 characters. The corpus comes with 2,010 questions, each tied to one novel. Figure 3.1 shows how the questions divide among the four types.

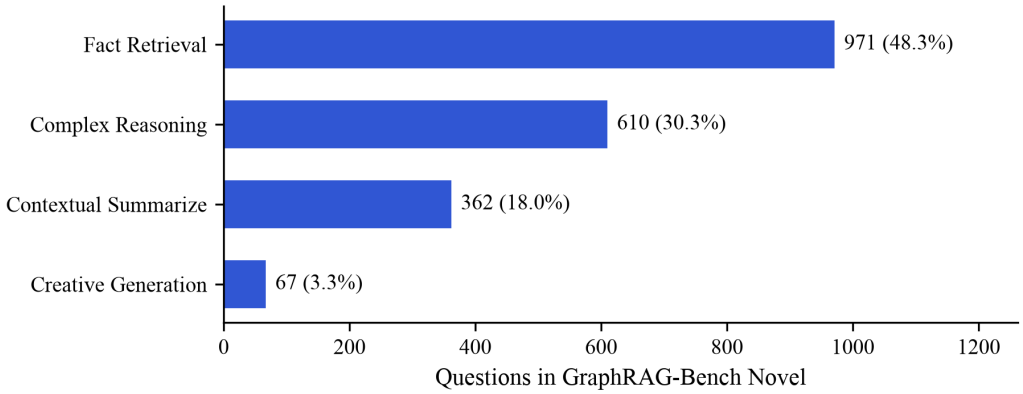


Figure 3.1: Question types in GraphRAG-Bench Novel

Each question carries a reference answer and two forms of gold evidence: a list of evidence sentences and a list of evidence triples of the form (head, relation, tail). An example from the corpus asks which plant, known scientifically as *Erica vagans*, is also referred to by another common name; the answer is "Cornish heath", and the gold triple is (erica vagans, is also known as, Cornish heath).

Before designing the gold-evidence analysis, we checked how the evidence sentences relate to the source text. They turned out to be restatements rather than extracts. Only 0.1% of them appear in their novel as exact substrings, and 5.4% reach a character n-gram similarity of 85 or more against windows the size of a text unit. Linking evidence by locating sentences in the text, as originally planned, was therefore not workable. We linked the triples instead, as described in Section 3.6 and Section 5.4.

3.4 FEASIBILITY AND CONSTRAINTS

The original plan was to build a GraphRAG index of the corpus with a language model and then measure loss and answer quality on it. We estimated the cost before starting. Indexing the roughly 1.1 million tokens of the corpus needs between 3,000 and 5,500 model calls for extraction, description summarization and community reports. The full answer-level study, including 125 questions under global search and the injection experiments, needs about 6,000 to 10,000 calls and on the order of 150 million tokens, most of them spent by global search.

The plan had to change for two reasons. On a free tier of the model service we had chosen, a daily request limit would have turned a long run into a stall lasting several days, so that tier could not handle the volume. The more serious problem was that the team's API key failed validation when we tried to start; its length and format were not what the service expected, and we stopped before spending any tokens.

We decided to work around this and carry on. A large part of the study needs no model, because GraphRAG's fast pipeline builds the graph, the communities and the summary inputs on its own, and everything that decides what a summary can contain is settled before the summarizer is ever called. A measurement of the summary inputs is therefore an exact measurement of the loss. Only the question of whether the loss changes answers needs a model, and we specified that test completely, freezing its thresholds in advance, so it can be run once access is available.

3.5 ANALYSIS OF THE GRAPHRAG 3.1.1 SOURCE

Since the design depends on what GraphRAG 3.1.1 really does, we read the relevant parts of its source before writing any measurement code. Everything in the list below was checked against the installed package.

- Text units default to 1,200 tokens with 100 tokens of overlap, counted with the `o200k_base` encoding.
- `max_input_length`, the summary input budget, is 8,000 tokens unless changed.
- The standard pipeline sorts relationships by combined degree, highest first, and keeps admitting them until the next one would overflow the input; admission ends at the first item that does not fit.
- In the fast pipeline the summary input is made of the text units that support the community's relationships. These are sorted by the summed degree of the community's entities in each unit, and the same first-overflow rule applies.
- When a parent community's input overflows, its report is written from its children's reports instead of its own content. A leaf community that overflows is cut.
- At a given level, global search reads for each entity the report of the community with the largest identifier among those at or above that level, and on every census graph we checked that this was always the deepest such community.
- Before clustering, the fast pipeline prunes the graph. It drops entities seen fewer than twice, edges whose weight is below the 40th percentile, and the single most connected entity.
- Clustering is hierarchical Leiden with a maximum community size of ten and a fixed default seed.

We then compared the package with our own design notes, which had been written from GraphRAG's documentation and paper, and found seven differences, listed in Table 3.4. For each one we adopted the same rule: detect it at run time and log it, carry on if it does no harm, and stop if it breaks an assumption that the measurement relies on.

Table 3.4: Differences between the documentation and GraphRAG 3.1.1

Documented	Observed in 3.1.1	Severity
Text units list relationships in a column spelled `relationships_ids`	The column is `relationship_ids`; the misspelling no longer exists	High
Documents carry a `metadata` column	Documents carry `creation_date` and `raw_data` instead	Low

Documented	Observed in 3.1.1	Severity
No mention of readable identifiers	Every file carries a `human_readable_id` column	Low
No mention of extra columns	Communities add `period`; text units add `covariate_ids`	Low
Gold evidence sentences can be found in the text	Evidence sentences are restatements (0.1% exact matches)	High
Relationship endpoints are entity identifiers	Endpoints are entity names; communities list entity identifiers	High
The tokenizer is not specified	`o200k_base` is the default encoding	Medium

Of the seven, the sixth was the most dangerous, since it produces no error at all. A check comparing relationship endpoints with the entity identifiers listed in communities would match nothing and still pass, having tested nothing. We only caught it because our tests ran against a synthetic index that copied GraphRAG's real schema, where a simplified schema would have hidden the problem.

3.6 PRE-REGISTERED EVALUATION DESIGN

Whether the lost content changes answers is a question that is easy to get wrong once the results are in, because a small effect tempts one to lower the bar for what counts as an effect. We guarded against this by writing down four kill criteria before downloading any corpus or building any index, and committed them to the project's git repository on 15 August 2026 (commit `d223de8`). A second commit the same day (`21e43d8`) added the linking parameters and the tokenizer. The file that holds them is treated as write-once.

Each criterion is a gate. If a gate fires, the claim it protects is dropped, later experiments do not run, and the result is reported as a null. Table 3.5 lists the gates.

Table 3.5: Kill criteria committed before any data existed

Gate	Experiment	Question and measurement	Stop if
G1	E5	Do community summaries already paraphrase the relationships they leave out of scope? Measured by NLI on 300 edges stratified by weight, once two annotators agree at Cohen's kappa of at least 0.7	More than 70% are entailed
G2	E6	Do benchmark questions need the lost content? Share of questions whose gold relationships intersect the loss set	Below 10%
G3	E8	Are wrong answers caused by the index rather than by retrieval or generation? Share of failures attributed to index absence	Below 15%

Gate	Experiment	Question and measurement	Stop if
G4	E9	Does restoring the lost content change answers? Oracle minus placebo accuracy, paired by question	The 95% CI includes zero

Two parts of the design deserve explanation. Gate G4 compares two injections of identical token count: the actual lost gold evidence, called the oracle, and randomly chosen lost relationships of the same size, called the placebo. A plain before-and-after comparison cannot separate the effect of the content from the effect of a longer context, which is a known source of change in model behaviour (Liu et al., 2024). The second part concerns gate G2. The frozen linking rule requires both the head and the tail of a gold triple to reach a `token_set_ratio` of at least 85 against an entity title of the question's own novel. A further rule, frozen at the same time, says that if more than 15% of triples fail to link, G2 is reported as inconclusive rather than as a pass or a fail. We apply that rule in Chapter 6.

The commit history is the evidence that the thresholds came first: the commits that froze them precede every commit that produced a number. The same file also records thresholds for a strong result, for example an oracle advantage of more than five accuracy points, so that the meaning of "strong" could not be adjusted once results were in.

Chapter 4

System Design and Methodology

This chapter describes how the project is put together and why. It begins with the overall architecture and the order in which the work was done, then develops the model of index-time loss that the measurements rest on, and then presents Residual Graph Indexing in full. The last two sections describe the design of the measurement toolkit and of the GraphRAG X-Ray simulator.

4.1 SYSTEM ARCHITECTURE

The project has three layers, shown in Figure 4.1. The first measures and repairs loss on real data without a language model. It takes the GraphRAG-Bench Novel corpus and GraphRAG 3.1.1's own code as inputs, runs the loss census, the selection analysis and the RGI builder, and writes result tables. An analysis script turns those tables into every figure, table and number that the paper and this report quote.

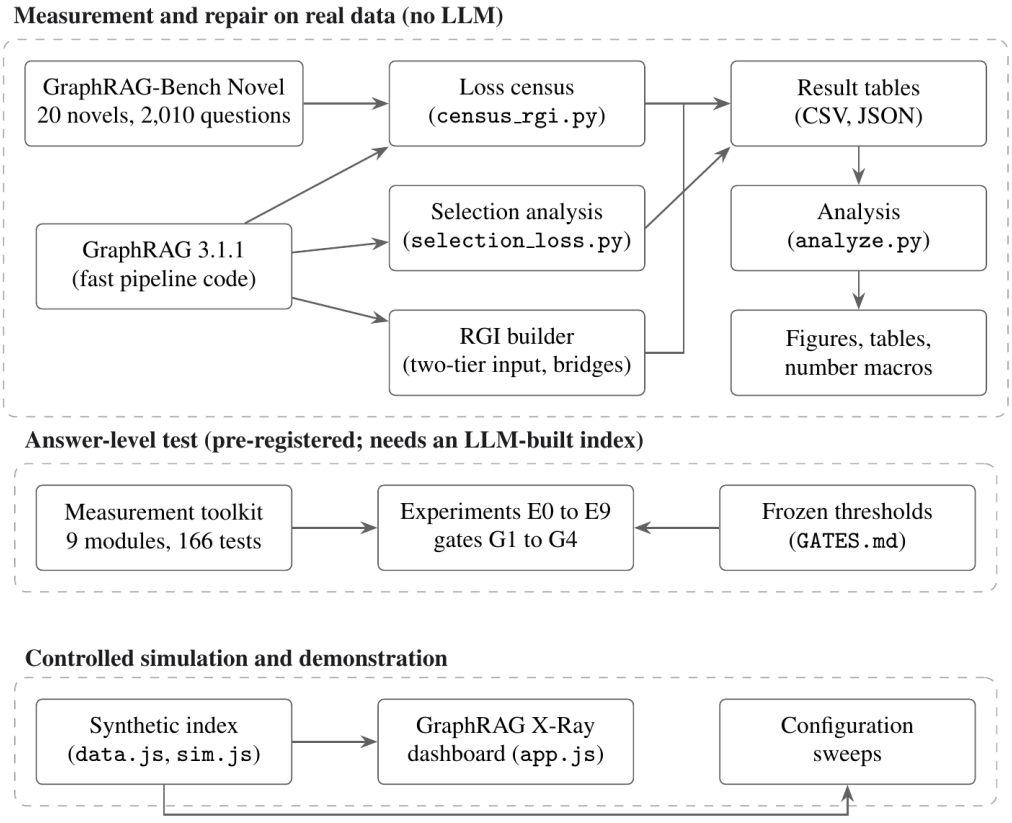


Figure 4.1: System architecture of the project

The second layer is the pre-registered answer-level test. It consists of a tested measurement toolkit, ten experiment scripts and the frozen thresholds described in Section 3.6. This layer needs an index built with a language model, so it is implemented and tested but not yet run. The third layer is a controlled simulation, GraphRAG X-Ray, which models a small index in the browser, shows the mechanism visually, and lets us carry the loss through to retrieval and to judged answers.

The layers share definitions rather than code. The loss model in Section 4.3 fixes what "lost" means, and all three layers count loss the same way: by asking, for each relationship, whether it reaches a summary input that is about its own community.

4.2 METHODOLOGY AND WORKFLOW

Figure 4.2 shows the order in which the work was done. The order matters, because several later decisions depended on earlier ones.

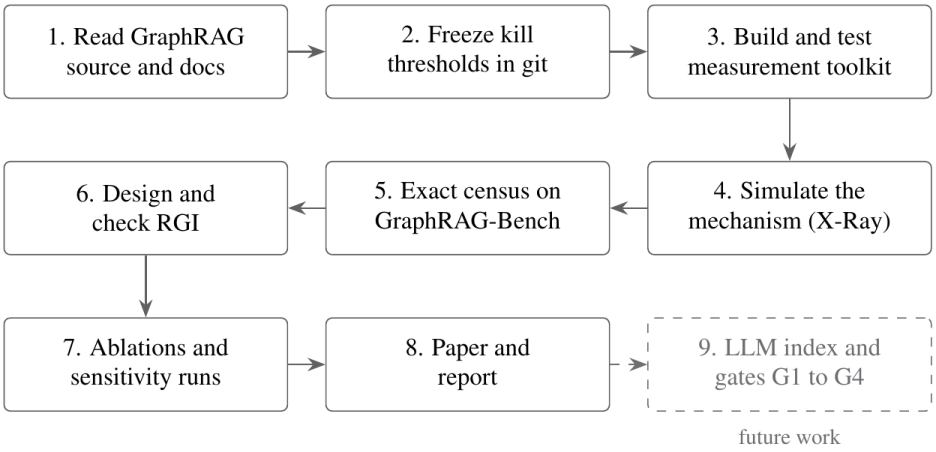


Figure 4.2: Project workflow

We began by reading GraphRAG's source code and documentation (Section 3.5), because the loss mechanisms are properties of specific functions. The kill thresholds were committed next, before any data existed. The measurement toolkit came third, built and tested against a synthetic index. With the tooling in place we built the GraphRAG X-Ray simulation, which made the mechanism concrete enough to reason about and to demonstrate. When it became clear that the language-model index could not be built in time, we moved the main measurement to GraphRAG's fast pipeline, which allowed an exact census on the real corpus. RGI was designed against the simulation and then checked against the census. Ablations and sensitivity runs

followed, and the results were written up as a research paper and as this report. Running the answer-level gates on a language-model index is the step that remains.

4.3 MODEL OF INDEX-TIME LOSS

4.3.1 NOTATION

The corpus is split into a set of text units U. Extraction produces a knowledge graph G = (V, E), where V is the set of entities and E the set of relationships. Each relationship r = (a, b) has a support U(r), the text units it was extracted from. Clustering produces a set of communities C. Each community c has a level λ(c), with level 0 the coarsest, and the hierarchy is nested: the children of a community divide its entities between them.

Global search at level ℓ reads the summaries of a selected set of communities. For every entity v, it uses the deepest community that contains v and lies at or above level ℓ:

$$\sigma\ell(v) = \text{the deepest community } c \text{ with } v \in c \text{ and } \lambda(c) \le \ell, \quad S\ell = \{\, \sigma\ell(v) : v \in V \,\}$$

An entity that was never clustered has no σℓ(v). Each community c has a summary input I(c), built by an input builder under the budget B, and a summary written by a model from that input. We write T(x) for the number of tokens in a text x under the `o200k_base` encoding, so that T(I(c)) ≤ B. A relationship is *in scope* for a community when both of its entities belong to that community. It is *represented* in an input when its record appears there (standard pipeline) or when one of its supporting text units does (fast pipeline). The List of Symbols at the front of this report collects all of this notation.

4.3.2 STATUS OF A RELATIONSHIP

At a level ℓ, every relationship r = (a, b) receives exactly one of the six statuses in Table 4.1. The first two depend only on the clustering; the last four depend on how the input of the shared community was built.

Table 4.1: Status of a relationship r = (a, b) at level ℓ

Status	Condition
Outside	a or b was never clustered, so it has no community at all
Boundary	a and b are read through different communities, σℓ(a) ≠ σℓ(b), so r is in neither community's scope

Status	Condition
Represented	a and b share a community c, and r is represented in I(c)
Sub-summary	a and b share a community c whose input was assembled from its children's summaries rather than its own content
Selection	a and b share a community c, but the input builder never offered r as a candidate
Budget	the builder offered r but cut it when the input reached B

A relationship is out of scope at level ℓ whenever its status is anything other than represented. We write $L\ell$ for the set of relationships out of scope at level ℓ. Figure 4.3 places these mechanisms on GraphRAG's pipeline, together with the components that RGI adds.

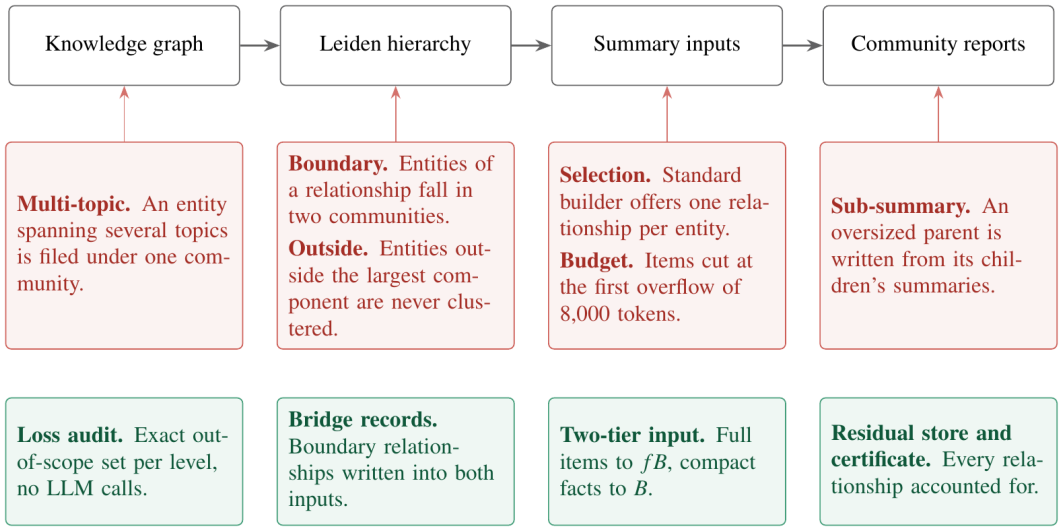


Figure 4.3: Where content leaves the path to an answer (red) and what RGI adds (green)

A seventh kind of loss works at the level of entities rather than relationships. An entity whose text units cover several topics is filed under one community and read in that community's framing only. We call this multi-topic loss. It does not remove the entity from the index, since global search still reads it, so it falls outside the propositions below.

4.3.3 SCOPE AND EVIDENCE

Scope is a property of the index structure, but it does not settle whether a relationship's evidence reaches a summarizer. In the fast pipeline, a summary input is a set of text units, and a single unit of 1,200 tokens can mention dozens of entities. A relationship can be out of scope for its own communities while the text that supports it

sits in some other community's input. In the standard pipeline, a description of an entity can mention a relationship that has no record of its own in the input.

We therefore grade every relationship by the strongest way its evidence reaches a selected input, using the four tiers in Table 4.2, from strongest to weakest.

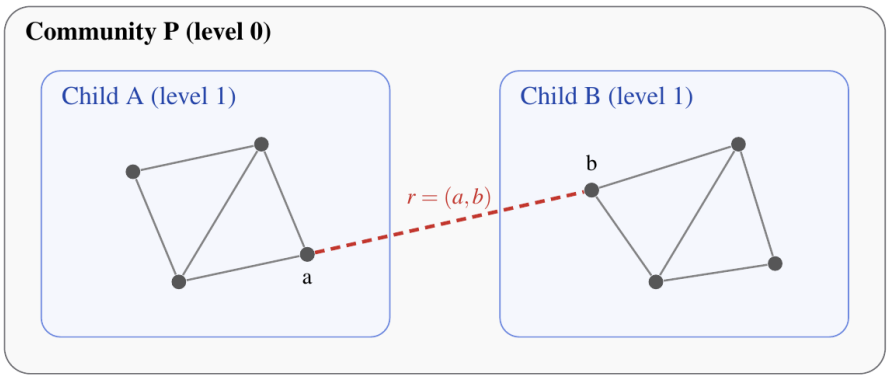
Table 4.2: Evidence tiers

Tier	How the evidence of r reaches the selected inputs
In scope	r is represented in the input of its own community
Endpoint	a supporting text unit sits in the input of a community that holds one of r's two entities
Elsewhere	a supporting text unit sits only in inputs of communities that hold neither entity
Absent	no record of r and no supporting text unit appears in any selected input

The tiers matter for what can be claimed. A relationship in the endpoint tier may still be mentioned by a summary, because the summarizer of an endpoint's community sees the text. A relationship in the absent tier cannot be, which is the subject of the first proposition below.

4.3.4 THE HIERARCHY TRADE-OFF

Moving up the hierarchy merges communities, so fewer relationships cross a boundary. It also makes each community larger, so more content has to pass through the same budget. Figure 4.4 illustrates the trade-off with two children of one parent community. At the child level, the relationship r between a and b crosses a boundary and belongs to neither child. At the parent level r is in scope, but the parent's input now has to carry everything from both children, and if it overflows, the parent is summarized from the children's summaries instead of its own content. The level that is chosen therefore changes the mechanism through which r is lost, and in this example r can be lost at either level.



At level 1 the relationship r joins entities in A and B, so it is in neither child's scope (boundary loss). At level 0 it is in scope of P, but P's input must now carry everything in A and B through the same 8,000-token budget, and if it overflows, P is written from the summaries of A and B instead of its own content.

Figure 4.4: Boundary loss at a fine level and compression at a coarse level

4.4 THREE PROPOSITIONS

The model supports three statements that hold for any corpus, beyond the one we measured.

Proposition 1 (absent evidence is unreachable). Fix a level ℓ and any retrieval policy whose context for a question depends only on the question and the summaries of the communities in $S\ell$. If the evidence of a relationship r is absent at level ℓ, the context does not change when the text units that support r are edited, for any question.

Argument. Each summary depends on the corpus only through its input. If r's evidence is absent, no supporting unit and no record derived from one appears in any input of a community in $S\ell$. Editing those units therefore leaves every input, every summary and every context unchanged.

We kept this proposition narrow on purpose; it applies to absent evidence and to nothing else. A relationship can be out of scope and still have its text in some input, and in that case it may surface in a summary anyway. How often that happens is what gate G1 of the pre-registered test is meant to measure.

Proposition 2 (boundary loss grows toward the leaves). Take a nested hierarchy and two levels $\ell < \ell'$. Any relationship whose status is boundary at level ℓ is still a boundary relationship at level ℓ'.

Argument. In a nested hierarchy, two different communities are either disjoint or nested one inside the other. Suppose $\sigma\ell(a)$ strictly contained $\sigma\ell(b)$. Then $\sigma\ell(a)$ would have a child, at a level no deeper than ℓ, that contains a, and $\sigma\ell(a)$ would not be the

deepest community holding a, contrary to how it was chosen. The two communities must therefore be disjoint. Going down to a deeper level ℓ' can only replace each of them with one of its own descendants, so they remain disjoint and r is still a boundary relationship.

Proposition 3 (selection bound). Whatever the budget, GraphRAG 3.1.1's standard input builder can represent no more than $|c|$ of the relationships in scope for a community c. It follows that at most $2/\bar{d}$ of c's internal relationships are represented, where $\bar{d}$ is the mean number of internal relationships per entity of c.

Argument. Inside the builder, the function `_prepare_reports_at_level`, a level's relationships are grouped once by source entity and once by target entity. Only the first relationship of each group is kept before the two results are merged, so no entity contributes more than one relationship record and the input can offer at most $|c|$ records. With $|c| \cdot \bar{d}/2$ internal relationships in c, the share cannot be higher than $2/\bar{d}$.

In practice the share is lower than the bound, for two reasons: the single record an entity contributes may point outside its community, and two entities can contribute the same record. On a sparse graph, with $\bar{d}$ close to 2, the bound barely matters, while on a dense graph it removes most of the relationships. GraphRAG's issue tracker has an independent report of the same behaviour (Grenat, 2026), and Section 6.4 puts numbers on it.

4.5 RESIDUAL GRAPH INDEXING

4.5.1 DESIGN CONSTRAINTS

Four constraints applied to RGI from the start. The first two keep the comparison fair. GraphRAG's clustering, summary prompt and global search are left exactly as they are, and so is the token budget of every summary input and every query context, because extra tokens alone change how a model behaves (Liu et al., 2024). With both fixed, any difference in results has to come from what the summary inputs contain. The other two are about cost and accounting: RGI may not add any language model call at indexing time, and it has to be able to report, for every relationship in the graph, where that relationship ended up. Behind the design is the video-coding idea from Section 2.7. The community summaries are the keyframes, which GraphRAG writes as usual, and RGI supplies the residual.

4.5.2 LOSS AUDIT

The loss audit, the first component, works out every relationship's status at every level. Finding boundary and outside relationships only takes a set difference on GraphRAG's own tables, all relationship identifiers minus the ones listed inside the selected communities. For the budget and selection statuses the audit replays the input builder with GraphRAG's own sort order, budget and tokenizer, and since the builder is deterministic the replay matches it exactly. No model is called, and apart from the cost of counting tokens the running time grows linearly with the number of relationships at each level.

4.5.3 TWO-TIER SUMMARY INPUT

In GraphRAG, items enter the input in priority order until one of them would overflow B, and everything from that item onward is dropped together. RGI works with the same items in the same order and changes only what happens in the last part of the budget, as the comparison in Figure 4.5 shows.

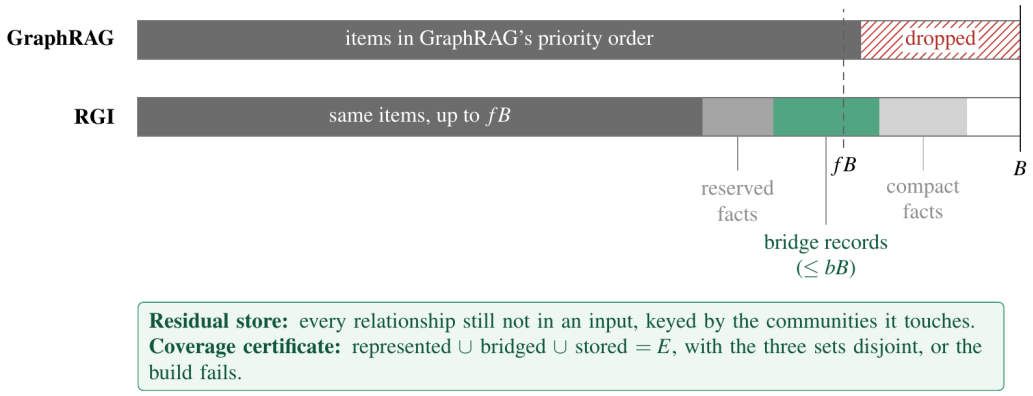


Figure 4.5: How GraphRAG and RGI fill the same summary input

If a community's input already fits within B, RGI keeps all of it and spends any slack on bridge records. If it overflows, full items are kept only up to a cutoff fB (f = 0.8 by default), and the remaining space is filled with one-line compact facts written as `source,target,support`. At about ten tokens each, compared with roughly 1,200 for a text unit, compact facts let the freed space hold many more relationships. Something is lost in the exchange, since a compact fact records that two entities are related and gives no detail about the relation itself.

One more rule is needed for monotonicity. Stopping the full items at fB instead of B can drop relationships that GraphRAG's own input did represent, so RGI writes

compact facts for exactly those relationships before anything else. If even these reserved facts will not fit, RGI uses GraphRAG's own input for that community and spends only the slack that is left. Communities whose input is built from their children's summaries are handled in a similar way: the child summaries are limited to fB, and the tail holds compact facts for the community's own relationships.

4.5.4 BRIDGE RECORDS

Bridge records can take up to a share b of B in each community, 0.12 by default. A bridge record is a boundary relationship with at least m supporting text units (m = 2 by default). These relationships are ranked by combined degree and written as compact facts into the inputs of both endpoint communities, and a relationship counts as bridged if it lands in at least one of the two. When GraphRAG has already extracted a description of the relationship, the bridge record can carry that description, so no summary has to be rewritten and no model is called.

4.5.5 RESIDUAL STORE AND TOKEN-EXACT SUBSTITUTION

Every relationship that is neither represented nor bridged goes into a residual store, keyed by the communities it touches, or by a global key for relationships whose entities were never clustered. At query time, after global search has chosen its reports, the stored facts of the chosen communities that best match the question can be added to the context. They replace the lowest-ranked part of the context, so that the token count stays exactly the same. The toolkit's implementation of this swap is described in Section 5.2.

4.5.6 COVERAGE CERTIFICATE

At build time, RGI checks for every level that the represented, bridged and stored sets are disjoint and that together they make up the whole relationship set E. If the check fails, the build stops. GraphRAG today discards content without any record. The certificate makes the discard visible and bounded, and it would also catch loss from a mechanism nobody anticipated, such as the selection behaviour of Proposition 3.

4.5.7 ALGORITHM

The way RGI builds one community's summary input is drawn in Figure 4.6, and the listing below spells out the same procedure step by step. In it, P is GraphRAG's list of items for the community in GraphRAG's own order, k_G is how many of those

items GraphRAG admits under B, E_in and E_bd are the in-scope and boundary relationships, and rep(X) is the set of relationships represented in an input X.

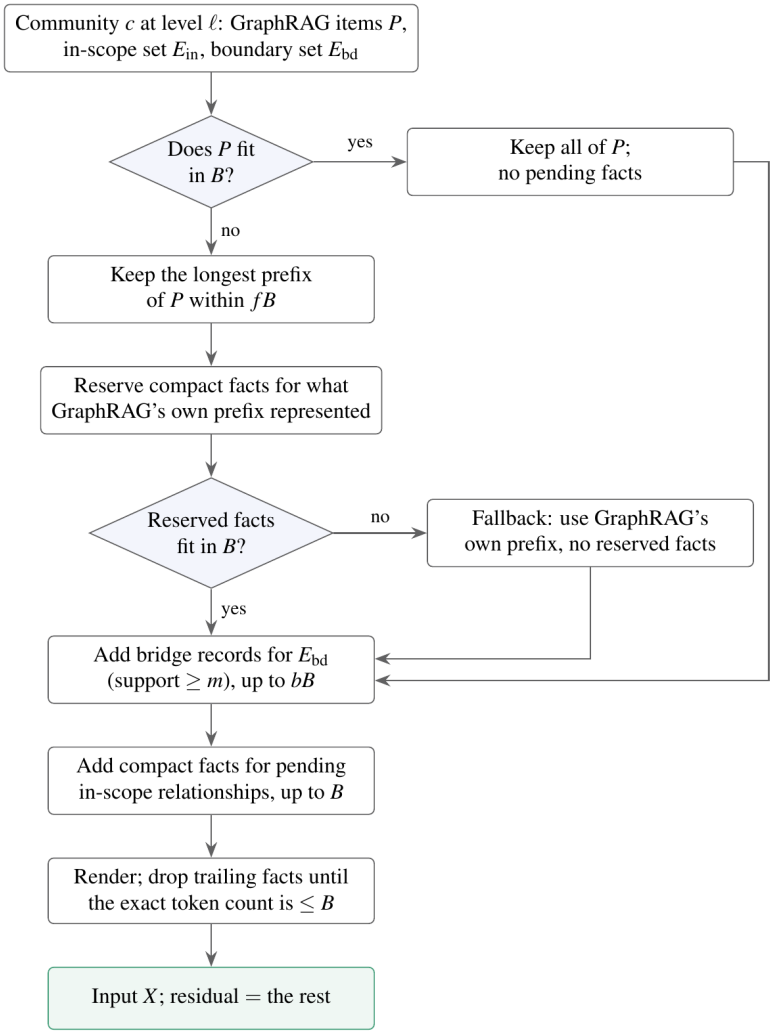


Figure 4.6: Construction of one RGI summary input

```
Input: items P in GraphRAG order; in-scope set E_in; boundary set E_bd;
       budget B; cutoff f; bridge share b; minimum support m
1   k_G  <- number of items GraphRAG admits under B
2   if T(P) <= B:  X <- P;  pending <- empty
3   else:
4       k <- longest prefix of P with T(P[1..k]) <= f*B          # k <= k_G
5       Q <- rep(P[1..k_G]) minus rep(P[1..k]), by combined degree   # reserved
6       if T(P[1..k] + facts(Q)) > B:  k <- k_G;  Q <- empty         # fallback
7       X <- P[1..k] + facts(Q);  pending <- E_in minus rep(P[1..k_G])
8   for r in E_bd with |U(r)| >= m, by combined degree:
9       stop when bridge tokens exceed min(b*B, B - T(X)); else append bridge(r)
10  for r in pending, by combined degree:
11      stop when T(X + fact(r)) > B;  append fact(r)
12  while T(render(X)) > B:  drop the last compact fact
13  residual(c) <- (E_in + E_bd) minus rep(X)
14  return X
```

4.5.8 GUARANTEES

Write R(G, ℓ) for the relationships that are in scope and represented under GraphRAG at level ℓ, and R(RGI, ℓ) for the same set under RGI. For every community and every level, RGI guarantees four things.

No input exceeds the budget, so T(I_RGI(c)) ≤ B in every case. Step 12 checks the bound on the rendered text, using the tokenizer that GraphRAG itself uses.

Monotonicity holds: R(G, ℓ) ⊆ R(RGI, ℓ). When the input fits, X contains all of P. When it does not, either the reserved facts bring back every relationship that GraphRAG's prefix represented and the shorter prefix lost, or the fallback restores GraphRAG's own prefix, and bridges and later facts can only add to this. Because the reserved facts come before every other appended line, step 12 could reach them only if they overflowed by themselves. Step 6 rules that out except for token merges at line boundaries, and the census checks the result on every run.

The cost stays the same. Like GraphRAG, RGI makes one summarization call per community, because it changes what goes into each input and leaves the number of summaries alone.

Coverage is complete. Every relationship in E falls into exactly one of the represented, bridged and stored sets, because the residual store receives a relationship precisely when it is neither represented nor bridged.

Monotonicity, as stated, concerns relationships in scope. In an overflowing input RGI replaces GraphRAG's last full items with compact facts, which takes the prose of those items out of the input. A relationship that was visible only by chance in that prose can therefore lose its visibility, and the census counts how often this happens (Section 6.5).

4.6 DESIGN OF THE MEASUREMENT TOOLKIT

For the answer-level test the census is not enough, since the test also has to run global search, link gold evidence, classify failures and inject content into contexts. We designed a Python package for this work around two rules. Any function that reads GraphRAG's output has to work on a synthetic index whose correct answers are known, which lets the tests check exact sets instead of numbers that merely look plausible. And

any experiment with a precondition, for example annotator agreement or a token-exact injection, refuses to run until that precondition is met.

The package's structure is drawn in Figure 4.7. Shared foundations sit in four modules (column names and schema checks, input and output with run records, statistics, and report writing), and the measurements sit in five more: loss computation, gold-evidence linking, NLI-based recoverability, failure attribution and token-exact injection. Ten scripts, numbered E0 to E9, run the experiments in a fixed order, and each one writes a table and a short report. Each module is described in Section 5.2.

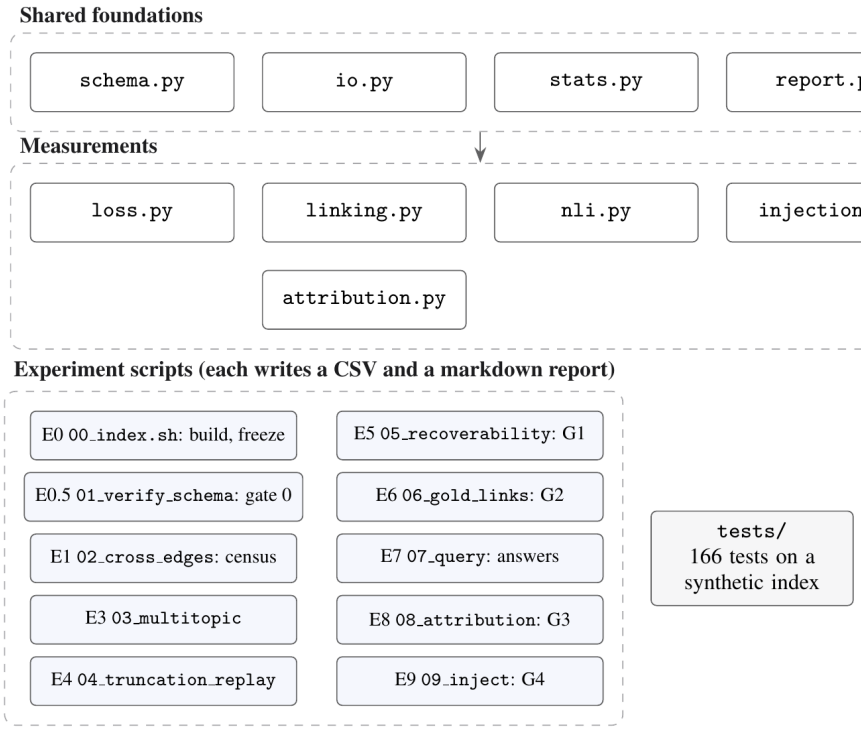


Figure 4.7: Structure of the measurement toolkit

4.7 GRAPHRAG X-RAY SIMULATOR DESIGN

GraphRAG X-Ray was built for two purposes: to make the mechanism visible, and to study its effect on answers in a setting we control. It models a synthetic index shaped like George Eliot's *Middlemarch*, with a core of 102 entities and 162 relationships taken from the novel's plot. A generated long tail of minor characters, places and things brings this up to 360 entities and 905 relationships, arranged in 22 leaf communities that nest into 8, then 3, and finally 1 community above them.

In the simulation, summary inputs follow GraphRAG's first-overflow rule and priority order, and the token costs and weights are generated. For each question,

retrieval reads the top three summaries, ranked by how many entities they share with the question, by a size prior and by noise that both methods share. A stochastic judge then marks the answer correct with probability 0.8 if all the evidence was retrieved, 0.45 if at least half of it was, and 0.15 otherwise. Of the 48 questions, 5 were written by hand from the novel; the others were sampled as paths of two to four relationships. The simulated RGI uses the two-tier layout on every input, which is a simpler variant of the algorithm above.

The simulator is split across three files, as Figure 4.8 shows. A data file holds the core of the corpus, a simulation file builds the full index and evaluates both methods at every level, and an interface file draws the graph and handles user interaction. Typed questions are accepted as well. The simulator matches the names in a typed question to entities, takes the shortest chain of relationships between them as the evidence, and evaluates that chain in the same way as the benchmark questions. Since the judge and the long tail are generated, the simulator's numbers illustrate the mechanism under our assumptions and should not be read as measurements on real data.

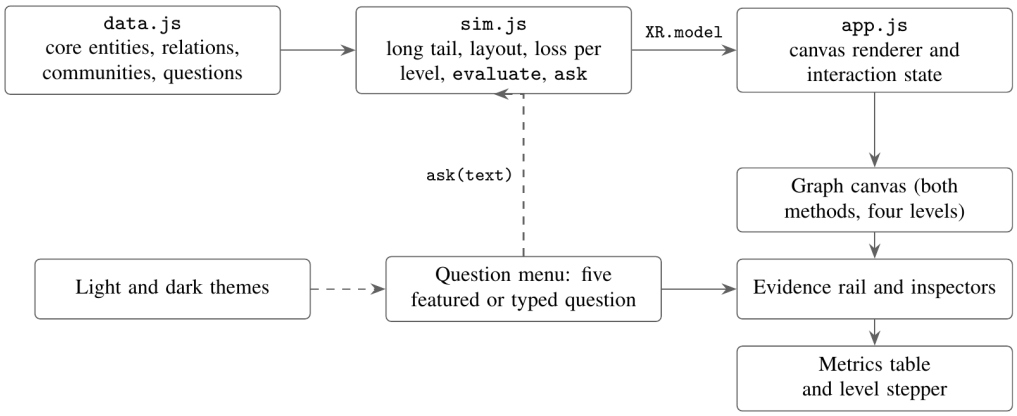


Figure 4.8: How the GraphRAG X-Ray simulator is organized

Chapter 5

Implementation

How the design was turned into working code is the subject of this chapter. We go through the repository and how it is laid out, the measurement toolkit, and the census pipeline behind the main results. After that come the linking of gold evidence, the analysis of GraphRAG's standard input builder and the scripts that turn results into figures and numbers, followed by the GraphRAG X-Ray simulator and the tests that check all of these.

5.1 REPOSITORY AND DEVELOPMENT ENVIRONMENT

Everything is kept in a single git repository, whose main folders are listed in Table 5.1. The Python code runs inside a virtual environment pinned by a lock file. The simulator is written in plain HTML, CSS and JavaScript, while the paper and the diagrams are in LaTeX. Any result in this report can be regenerated from the repository together with the public dataset.

Table 5.1: Main folders of the project repository

Folder	Contents
`src/graphrag_loss/`	Measurement toolkit: nine Python modules
`scripts/`	Ten experiment scripts, E0 to E9
`tests/`	166 tests and a synthetic GraphRAG index used as a fixture
`data/graphrag_bench/`	GraphRAG-Bench Novel corpus and questions (MIT licence)
`paper/experiments/`	Census, selection analysis, simulation sweep and analysis scripts
`paper/results/`	Result tables in CSV and JSON
`paper/generated/`	Generated LaTeX tables and number macros
`demo/`	GraphRAG X-Ray simulator
`GATES.md`	Kill thresholds and linking parameters, frozen before any data

5.2 MEASUREMENT TOOLKIT

The toolkit is a Python package with nine modules, and Table 5.2 gives a one-line summary of each.

Table 5.2: Modules of the measurement toolkit

Module	Responsibility
`schema.py`	Expected columns of every GraphRAG output file; detects and logs differences from the documentation
`io.py`	Loads a frozen index, hashes its files into a manifest, stamps every result with the git commit and GraphRAG version
`loss.py`	Boundary census per level and across all levels, multi-topic labelling, replay of the input budget
`linking.py`	Links gold evidence to graph elements using the frozen triple strategy
`nli.py`	Recoverability check with an NLI model, gated on human agreement
`attribution.py`	Classifies a failed answer as index-absent, not selected or not used
`injection.py`	Inserts content into a context while keeping its token count exactly constant
`stats.py`	Bootstrap intervals, McNemar's test, Cohen's kappa and h, partial correlation, Holm correction
`report.py`	Writes each experiment's report with fixed sections, including the counter-evidence

5.2.1 SCHEMA AND INPUT HANDLING

GraphRAG stores its index as parquet files, and the columns in those files have changed from one version to the next. For each file, the `schema` module records which columns the study expects and compares them with the columns it actually finds. A missing or renamed column is logged, after which the module either carries on, when the difference is harmless, or stops, when some measurement depends on that column; the seven differences in Table 3.4 were caught and recorded this way. The `io` module loads the files and turns list-valued columns into plain Python lists, which matters because parquet returns arrays and arrays behave differently when tested for truth. When an index is frozen, `io` also writes a manifest of file hashes. Each result file starts with a header naming the git commit, the GraphRAG version and the manifest hash, so any number can be traced back to the exact index and code that produced it.

5.2.2 LOSS COMPUTATION

Computing the boundary census is straightforward once one has GraphRAG's community table, since that table lists the relationships lying wholly inside each community. At a given level, a relationship is a boundary relationship if no community at that level lists it, which in code is a set difference:

```
def intra_relationship_ids(communities, level=None):
    rows = (communities if level is None
            else communities[communities.level == level])
    out = set()
    for rids in rows.relationship_ids:
        out.update(io.as_list(rids))
    return out

def cross_edges_at_level(communities, relationships, level):
    return set(relationships.id) - intra_relationship_ids(communities, level)
```

The same module computes the permanent cross edges, those that no level contains, and the share of total edge weight they carry. It also replays GraphRAG's budget rule for the standard pipeline, sorting relationships by combined degree and admitting them until the first overflow, so that the relationships lost to the budget can be listed one by one. A check called `verify_intra_community_edges` confirms on every index that each listed relationship really has both entities in its community. That check needs relationship endpoints, which are entity names, to be mapped to entity identifiers first; without the mapping it passes without testing anything, which is the sixth difference in Table 3.4.

5.2.3 LINKING, RECOVERABILITY AND ATTRIBUTION

The `linking` module turns gold evidence into graph elements. It implements two strategies. The frozen default links each gold triple by matching its head and its tail against entity titles with RapidFuzz's `token_set_ratio`, after lowercasing and removing punctuation, and accepts the triple only if both endpoints score at least 85. The sentence strategy is kept for a possible later comparison but is not used in any gate.

For gate G1, the `nli` module sets up the recoverability check. It pulls out the passages of a community report that mention each entity, forms a premise from the reports of the two communities that a relationship connects, and takes the relationship's description as the hypothesis. During calibration it writes a labelling sheet for two annotators, and it will not classify anything until their agreement, measured by Cohen's kappa, reaches 0.7. Gate G3 gets the same treatment from the `attribution` module, which sorts each failed answer into one of three categories in pipeline order and waits until the annotators agree.

5.2.4 TOKEN-EXACT INJECTION

Since gate G4 compares two contexts of exactly the same length, the injection code had to be correct. In `inject_with_report`, the new content goes first. The

original context pieces are then re-admitted in their original priority order until the token budget is reached, the first piece that does not fit is truncated, and the result is padded to the original count exactly. A difference of even one token raises an error:

```
target = token_len(base_context, tokenizer)
...
assembled = _pad_to_exact(assembled, target, tokenizer, leftover or " ")
result = InjectionResult(context=assembled, base_tokens=target,
                         new_tokens=token_len(assembled, tokenizer), ...)
if not result.is_token_exact:
    raise InjectionError("token-exact contract violated: "
                         f"base={result.base_tokens} new={result.new_tokens}")
```

This is harder than it sounds, because a tokenizer can merge characters across the point where two strings are joined, and the token count of the joined string is then not the sum of its parts. For that reason `_largest_fitting_slice` counts tokens on the joined text while it searches for the longest slice that fits, and `_pad_to_exact` adds or removes one token at a time until the counts agree. The module also samples placebo content of the same size, drawn from lost relationships that are not part of the gold evidence.

5.2.5 EXPERIMENT SCRIPTS

The experiments are run by ten scripts in a fixed order (see Table 5.3). Every script writes a comma-separated values (CSV) table along with a short markdown report. If a gate fires or a precondition fails, the script exits with code 2, which makes a sequence of runs stop at the first problem.

Table 5.3: Experiment scripts

ID	Script	Purpose	Gate
E0	`00_index.sh`	Build the index and freeze it with a manifest	
E0.5	`01_verify_schema.py`	Integrity checks on the frozen index	0
E1	`02_cross_edges.py`	Census of boundary relationships per level	
E3	`03_multitopic.py`	Multi-topic entities, controlled for degree	
E4	`04_truncation_replay.py`	Relationships lost to the input budget	
E5	`05_recoverability.py`	NLI recoverability after annotator calibration	G1
E6	`06_gold_links.py`	Gold evidence intersected with the loss set	G2
E7	`07_query.py`	Global search over 125 questions	
E8	`08_attribution.py`	Failure attribution after annotator calibration	G3
E9	`09_inject.py`	Oracle versus placebo at equal token count	G4

Where the order of steps matters, the scripts enforce it. The truncation replay will not start until a written reading of GraphRAG's summarization code is in place, and the injection script goes through its three phases strictly one after another: first the token-exact check, then the placebo arm and last the oracle arm.

5.3 CENSUS PIPELINE

The main results in this report come from the census pipeline. It is one script, which indexes every novel separately and then evaluates each relationship at four query levels and four budgets, once under GraphRAG and once under RGI. Its stages, and the GraphRAG function used at each stage, are drawn in Figure 5.1.

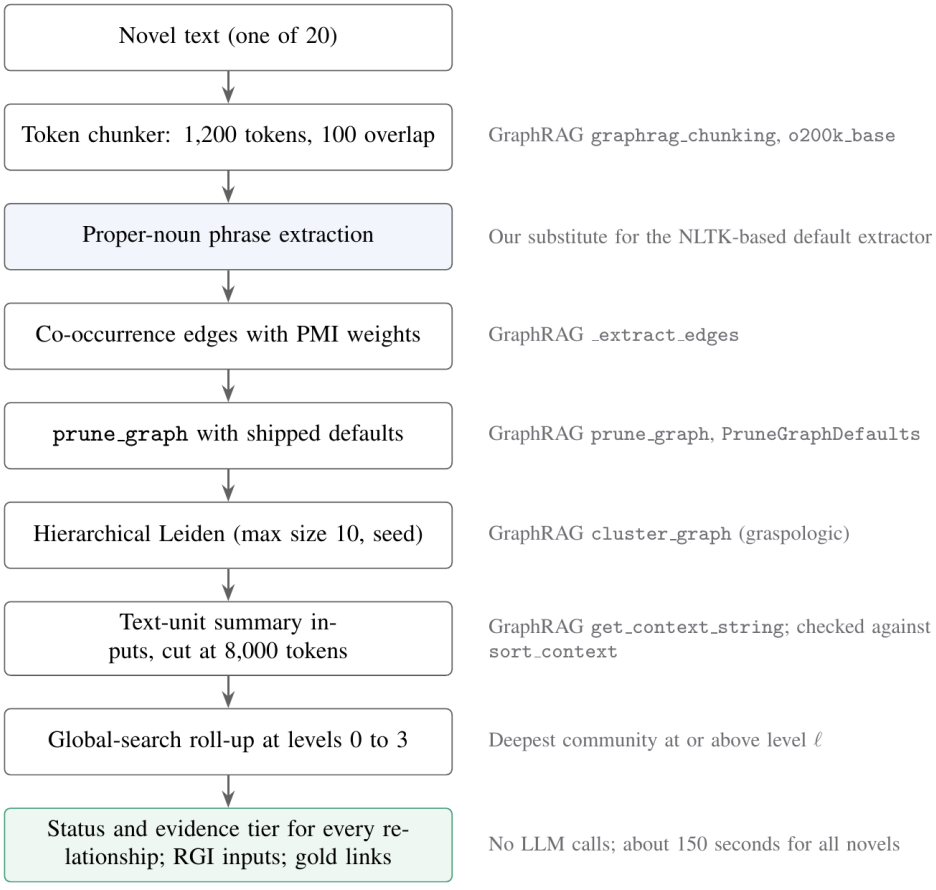


Figure 5.1: Stages of the census pipeline for one novel

5.3.1 INDEXING WITH GRAPHRAG'S OWN CODE

Chunking uses GraphRAG's token chunker with the default 1,200 tokens and 100 tokens of overlap. Edges come from GraphRAG's `_extract_edges`, which connects phrases found in the same text unit and gives each connection a PMI weight. GraphRAG's `prune_graph`, run with its shipped defaults, prunes the graph, and

GraphRAG's `cluster_graph` clusters it by calling hierarchical Leiden with a maximum community size of ten and the default seed. Then, for every community, the script gathers the text units of its internal relationships and sorts them by the summed degree of the community's entities in each unit, which is exactly what GraphRAG's `prep_text_units` does, before rendering them with GraphRAG's `get_context_string`.

Only one stage uses code that is not GraphRAG's. Its default noun-phrase extractor needs tagger data from the Natural Language Toolkit (NLTK), and that data could not be downloaded on the network we were using. In its place we wrote a deterministic proper-noun extractor with the same output format, the same stop words and the same rules for valid tokens as the default one. It collects runs of capitalized words, lets short connectors such as "of" sit between them and strips possessive endings. A capitalized word is treated as an ordinary word whenever the same word also occurs in lower case somewhere in the novel, and this rule gets rid of sentence-initial words like "The" or "Then" without any tagger. Wherever the results are reported, the substitution is stated.

5.3.2 EXACT ADMISSION

Under GraphRAG's admission rule, text units are added one by one, and the process stops at the first unit that pushes the rendered input over the budget. Re-rendering and re-tokenizing after each addition would be slow, so the script reaches the same cut by binary search over the number of admitted units, with token counts cached:

```
def admit(self, key, records, budget):
    n = len(records)
    if n == 0:
        return 0
    if self.tokens(key, records, n) <= budget:
        return n
    lo, hi = 0, n       # tokens(lo) <= budget < tokens(hi)
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if self.tokens(key, records, mid) <= budget:
            lo = mid
        else:
            hi = mid
    return lo
```

A binary search is only correct if the token count grows with the number of units, which holds here because each unit adds text. To be sure that the search returns what GraphRAG would return, the script calls GraphRAG's own `sort_context` on

the first overflowing communities of every novel and asserts that the admitted text is identical. The two agreed on all 139 communities checked.

5.3.3 STATUS, EVIDENCE AND RGI INPUTS

For each level, the script reproduces global search's choice of communities, takes the deepest community at or above the level for every entity, and asserts that this always equals the community with the largest identifier, which is the rule GraphRAG implements. It then assigns every relationship its status from Table 4.1. A community whose input fits keeps all its units; a leaf community that overflows is cut; a parent community that overflows is marked as assembled from its children's summaries.

The RGI builder follows the algorithm in Section 4.5 line for line. After building each input it renders the text, counts its tokens with the production tokenizer and drops trailing compact facts until the count is within the budget; an assertion then checks that no input exceeds B. After all communities are built, the script checks the coverage certificate, that the represented, bridged and stored sets are disjoint and cover every relationship, and counts any relationship that GraphRAG represented and RGI does not. A failed certificate stops the run.

Evidence tiers are computed last. The function below gives each relationship the strongest tier its evidence reaches:

```
def exposure(ix, sel, admitted, explicit):
    every = set().union(*admitted.values()) if admitted else set()
    out = {}
    for rid, src, tgt in zip(rel.id, rel.source, rel.target):
        if rid in explicit:
            out[rid] = "scope"
            continue
        units = ix["rel_units"][rid]
        near = (admitted.get(sel.get(src), set())
                | admitted.get(sel.get(tgt), set()))
        out[rid] = ("endpoint" if units & near
                    else "elsewhere" if units & every else "absent")
    return out
```

A full run over the twenty novels, including every budget, level and RGI configuration reported in Chapter 6, takes 149 seconds on the M2 laptop and makes no model calls. Two further runs repeat the main configurations with text units of 600 and 300 tokens.

5.4 LINKING GOLD EVIDENCE

For every question, the census script parses each gold triple into its head, relation and tail and links it against the entities of the question's own novel with the frozen configuration: `token_set_ratio` of at least 85 for both endpoints, applied to lowercased text without punctuation. A triple whose two endpoints link identifies the graph relationships between those two entities. The relationships found for all of a question's triples form its gold set Y(q). If two endpoints resolve to the same entity, or the two entities have no relationship in the graph, the triple contributes nothing. For each question and level, the script records how many gold relationships are in scope under GraphRAG and under RGI, and how many reach each evidence tier.

5.5 SELECTION IN THE STANDARD PIPELINE

The census measures the fast pipeline, whose input builder works on text units. To measure the standard pipeline's builder, which works on relationship records, a second script runs that builder unchanged on the census graphs. It calls GraphRAG's own `explode_communities`, `_prep_nodes`, `_prep_edges` and `build_local_context`, the same sequence GraphRAG's community-report workflow uses, and then reads which relationship records appear in each community's candidate pool. Descriptions are left as GraphRAG's own placeholder text, because the selection step never reads them. For each level, the script counts the within-community relationships that reach the pool, the cross-community relationships that enter pools, and the upper bound of one relationship per entity from Proposition 3, which it also asserts.

5.6 ANALYSIS AND REPRODUCIBILITY

No number in the paper or in this report was copied by hand. An analysis script reads the result tables and writes every figure, every table and more than 270 named numbers, which the paper's LaTeX source and this report's generator insert directly. When a result changes, rerunning the script changes every place it appears. Table 5.4 lists the commands that regenerate everything from the public dataset.

Table 5.4: Commands that regenerate the results

Command	Output
`python paper/experiments/census_rgi.py`	Census, RGI and gold links at 1,200-token units
`python paper/experiments/census_rgi.py --chunk-size 600 --no-ablations`	Sensitivity run at 600 tokens
`python paper/experiments/census_rgi.py --chunk-size 300 --no-ablations`	Sensitivity run at 300 tokens
`python paper/experiments/selection_loss.py`	Standard-pipeline selection counts
`node paper/experiments/sim_sweep.js`	Simulation results and configuration sweeps
`python paper/experiments/analyze.py`	Figures, tables and number macros

5.7 GRAPHRAG X-RAY SIMULATOR

GraphRAG X-Ray is a single web page that runs from local files in any modern browser. It uses no frameworks and loads nothing from the internet. Its three source files follow the organization in Figure 4.8.

5.7.1 BUILDING THE SIMULATED INDEX

The core of the corpus is kept in the data file. It contains the plot entities and relationships of *Middlemarch*, the 22 leaf communities with their titles and summaries, the parent communities above them, and the five hand-written questions, each with its evidence path and three versions of its answer. Everything else is built by the simulation file when the page loads. A long tail of minor entities is generated from name pools using a seeded random number generator, so each run gives the same index; token costs and weights are assigned, and the graph is laid out with a deterministic force-directed algorithm that we wrote from scratch.

At each of the four levels, and for each method, the simulation works out the status of every relationship. Under the original method, relationships are admitted in priority order until the first overflow. The RGI variant keeps full items up to 80% of the budget, uses up to 12% of the budget for bridge records of boundary relationships whose weight is four or more, and fills what remains with compact facts. Scoring is done by a function called `evaluate`, which takes a question, a level and a method, ranks the community reports, keeps the top three, checks which evidence relationships appear in them and then applies the stochastic judge.

5.7.2 INTERFACE

The graph is drawn on an HTML canvas, with lost relationships shown as red dashed lines, recovered ones as green lines and the evidence path in blue. For the current question, a comparison panel gives each method's verdict, and a stepper moves through the four levels while a table reports the share of relationships retained, the evidence retrieved and the answers judged correct. If the user clicks a relationship, an entity or a community, an inspector opens and explains in plain words why that element is lost or kept at the current level. Light and dark themes are both supported, as are keyboard shortcuts. Figure 5.2 and Figure 5.3 show one question at the leaf level under each of the two methods.

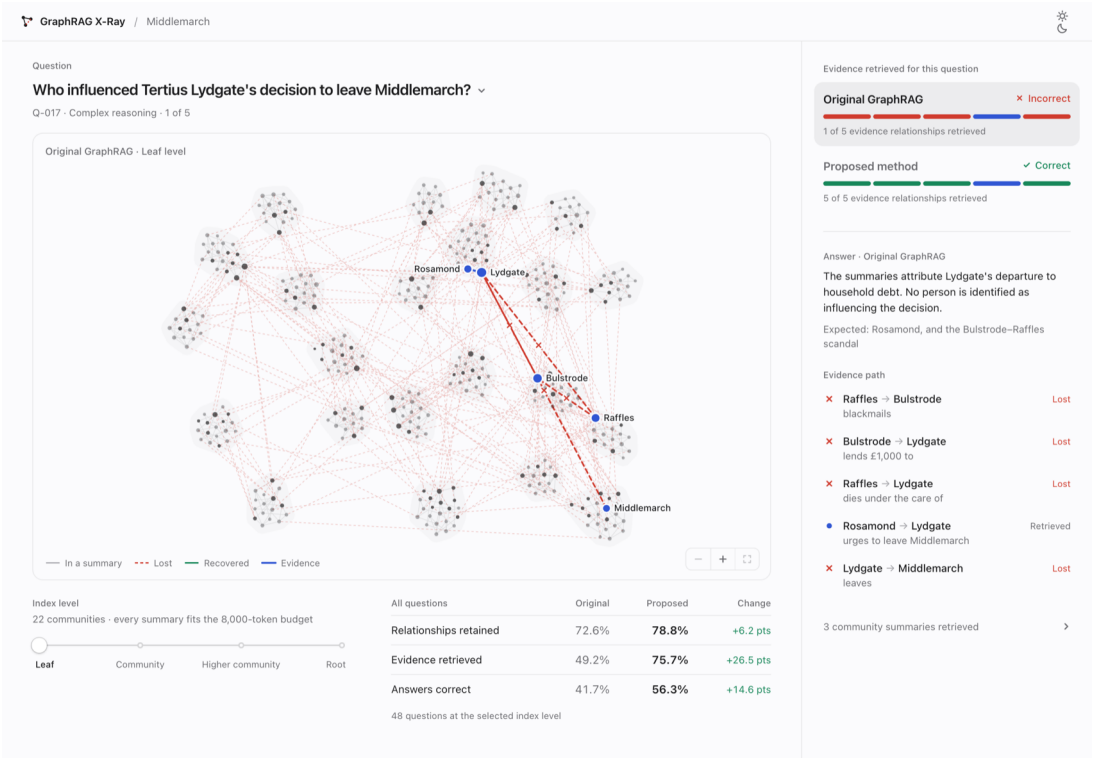


Figure 5.2: GraphRAG X-Ray: original GraphRAG at the leaf level

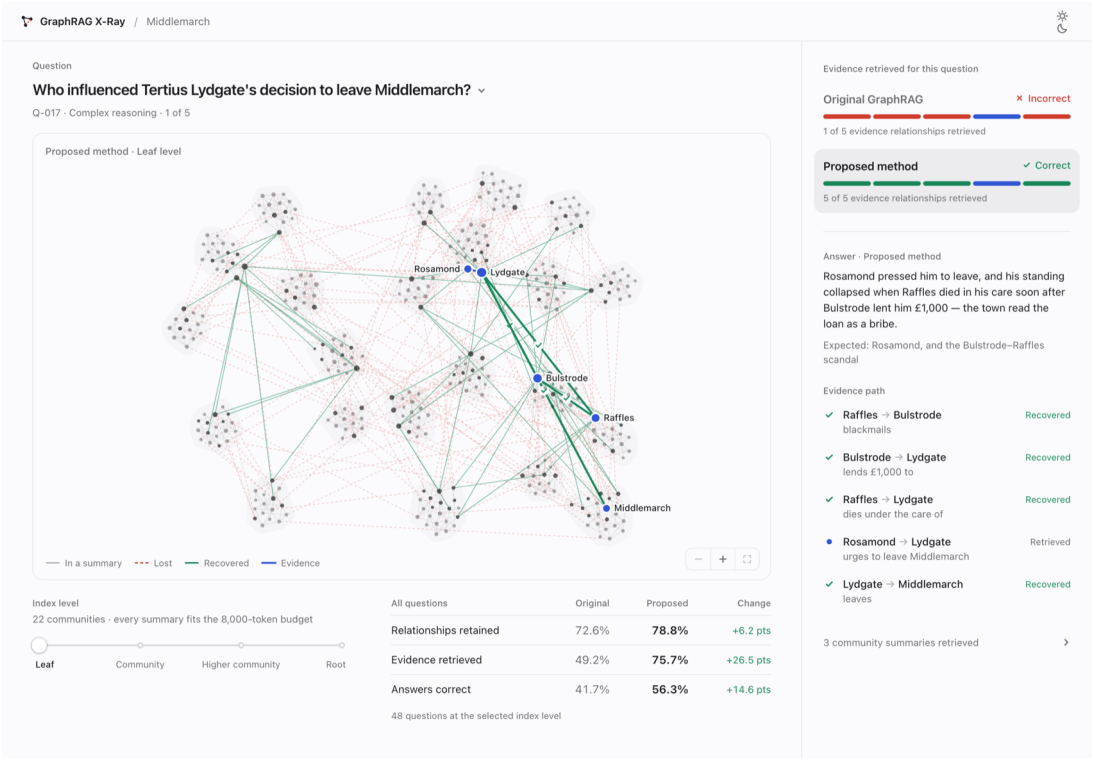


Figure 5.3: GraphRAG X-Ray: the same question under RGI

At the root level, shown in Figure 5.4, the single community's summary input overflows its budget, and the metrics table shows how much of the graph is retained by each method.

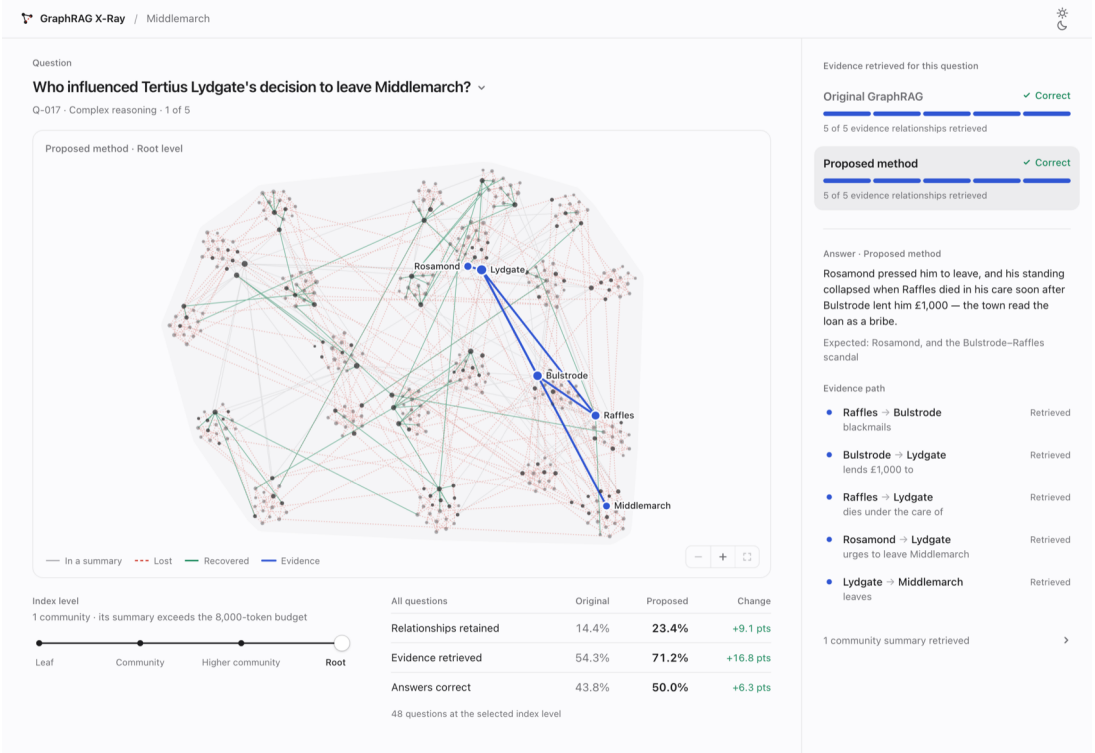


Figure 5.4: GraphRAG X-Ray at the root level under RGI

5.7.3 TYPED QUESTIONS

Beyond the five featured questions, a user can type any question about the novel. The function `ask` resolves the words of the question to entities by matching the longest phrases first against full names, short names and surnames, accepting plural and possessive forms, and breaking ties between entities with the same surname by preferring the most connected one:

```
function ask(text) {
  var raw = String(text == null ? "" : text).trim(), norm = normText(raw);
  if (!norm) return { ok: false, reason: "empty" };
  var toks = norm.split(" "), matched = [], seen = new Set(), i = 0;
  while (i < toks.length && matched.length < MAX_ENTS) {
    var hit = null, len = 0;
    for (var n = Math.min(4, toks.length - i); n >= 1; n--) {
      var gram = toks.slice(i, i + n).join(" "), a = alias.get(gram);
      if (!a && n === 1 && gram.length >= 5
          && gram.charAt(gram.length - 1) === "s")
        a = alias.get(gram.slice(0, -1));
      if (a) { hit = a; len = n; break; }
    }
    ...
```

When the question names two or more entities, the evidence is taken as the shortest chain of relationships linking them in order; when it names one, its strongest relationships from the novel's core are used. The resulting question goes through the same `evaluate` function as the benchmark questions, and the answer panel composes its answer only from the relationships that the method actually retrieved. Figure 5.5 shows a typed question.

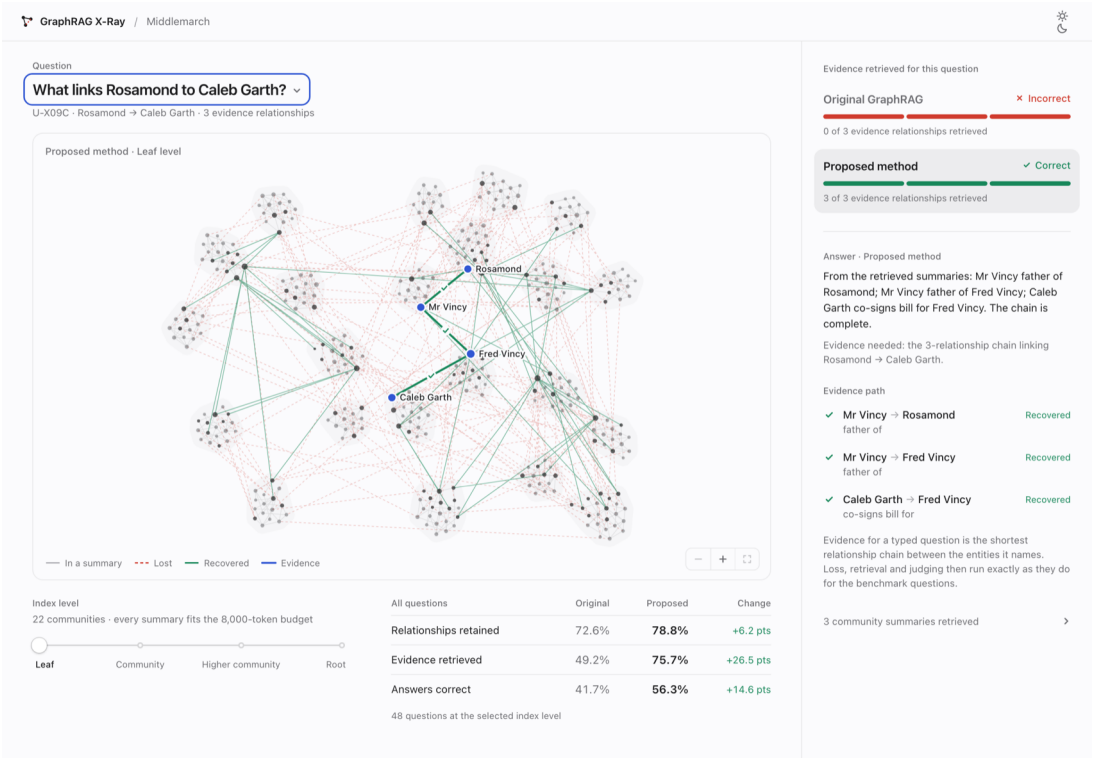


Figure 5.5: A typed question resolved to a three-relationship chain

5.8 TESTING

All 166 tests in the toolkit pass, and Table 5.5 shows how they are organized. They run against a synthetic GraphRAG index that is stored alongside the tests. This fixture follows GraphRAG's real schema, down to relationship endpoints being stored as entity names, and it is small enough for the correct answers to be worked out by hand. The boundary test is a good example: it asserts that the census returns exactly r6, r7, r8 and r9 at level 0 and exactly r7 and r9 at level 1, where a weaker test would only check that the result looked reasonable.

Table 5.5: Test suite of the measurement toolkit

Test file	Tests	What it checks
`test_injection.py`	75	Token-exact injection on 50 random contexts, 10 contexts with mixed scripts and punctuation, five insertion sizes; placebo sampling
`test_stats.py`	29	Bootstrap, McNemar and kappa against scipy and statsmodels
`test_schema.py`	23	Column checks and the relationship-column spelling in both directions
`test_linking.py`	23	Triple and sentence linking on known elements
`test_loss.py`	16	Exact boundary sets per level and across levels
Total	166	

The census pipeline also has checks of its own, run every time it executes instead of in a separate suite. Its admission rule has to agree with GraphRAG's `sort_context`, the global-search rule has to pick the deepest community, no RGI input may go over its budget, and the coverage certificate has to hold. We checked the simulation in the same way: at all four levels, the RGI variant lost no relationship that the original method had kept, and no question ended up with fewer evidence relationships retrieved. One project rule shaped all of this testing. If a test fails, work stops until the failure is fixed, and failures are never hidden inside an exception handler.

Chapter 6

Results and Discussion

The measurements and what they found are reported here. After the experimental setup, the chapter takes six questions one at a time. It asks how much each level of the hierarchy loses and how scope differs from evidence, then how GraphRAG's standard input builder selects relationships and how much RGI recovers within the same budget. After that it asks whether the lost content is what the questions need, and how sensitive the results are to the text-unit size and to RGI's parameters. Results from the controlled simulation come next, and a discussion and the threats to validity close the chapter.

6.1 EXPERIMENTAL SETUP

Each of the 20 novels was indexed separately with GraphRAG 3.1.1's fast pipeline at its shipped defaults, as described in Section 5.3. Table 6.1 summarizes the corpus and the resulting graphs.

Table 6.1: Census corpus and index

Quantity	Value
Novels / questions	20 / 2,010
Characters / tokens (o200k_base)	4,819,610 / 1,217,195
Text units (1,200 tokens, 100 overlap)	1,024
Extracted entities / relationships	14,825 / 533,441
After pruning: entities / relationships	4,831 / 116,217
Mean degree, pooled (median over novels)	48.1 (31.1)
Communities, all levels	1,226
Deepest level reached	5
Summary input budget	8,000 tokens
Largest novel's share of relationships	34%
LLM calls	0

The graphs vary a great deal in size, as Figure 6.1 shows. The largest holds 34% of all relationships, and the smallest has only 33. Because a pooled share is dominated by the largest graphs, every main result below is reported in two ways: pooled over all relationships, and as the mean of the per-novel shares, with a 95% bootstrap confidence

interval over novels. Each count is a census of every relationship, so no p-values are reported for it.

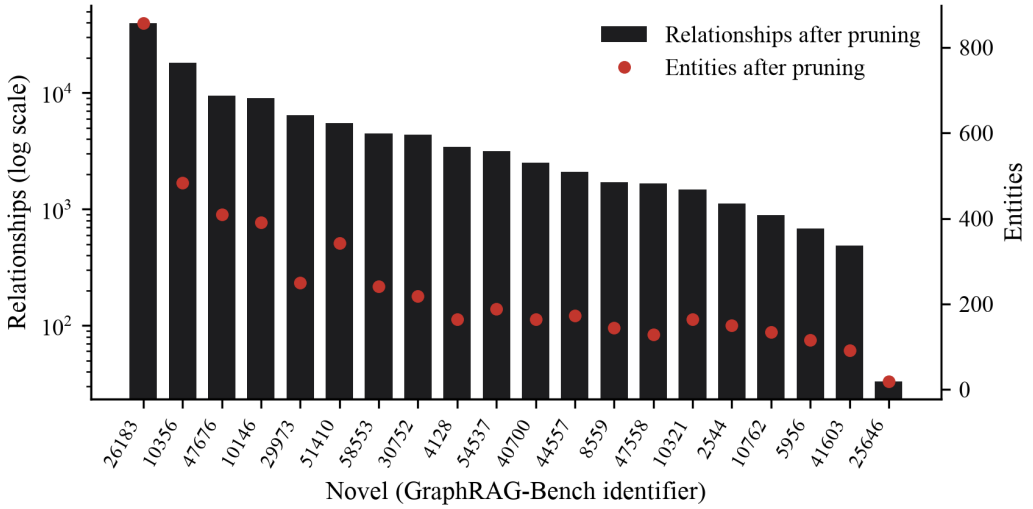


Figure 6.1: Size of each novel's graph after pruning

The query levels studied are 0 (the root), 1, 2 (GraphRAG's command-line default) and 3. Budgets of 4,000, 8,000, 16,000 and 32,000 tokens were run, with 8,000 the default. RGI ran with its default parameters, f = 0.8, b = 0.12 and m = 2, which were fixed before the census, and with seven variations of them.

6.2 HOW MUCH EACH LEVEL LOSES

Table 6.2 gives the status of every relationship at each level under GraphRAG, together with RGI's in-scope share and the mean gain over novels. At the default level 2, GraphRAG's summary inputs keep 12.4% of relationships in scope. Boundaries account for 84.9%. The budget accounts for almost nothing at this level, because nearly every selected community is a leaf of at most ten entities whose input fits within 8,000 tokens.

Table 6.2: Share of relationships (%) by status at B = 8,000

Level	Comm.	In scope	Boundary	Sub-summary	Budget	RGI in scope	Mean gain [95% CI]
0 (root)	123	1.9	47.2	50.9	0.00	12.4	+25.2 [16.3, 35.4]
1	389	7.8	71.8	20.4	0.00	27.4	+25.5 [20.6, 30.1]
2 (default)	687	12.4	84.9	2.6	0.01	34.6	+24.6 [20.5, 28.3]
3	797	12.1	87.9	0.0	0.01	35.6	+24.9 [20.9, 28.6]

The root level looks very different. There only 1.9% of relationships stay in scope, 47.2% cross a boundary, and 50.9% belong to communities whose summary inputs are assembled from their children's summaries in place of their own content. From the root down to level 3, boundary loss climbs steadily from 47.2% to 87.9%, as Proposition 2 predicts. Where each relationship ends up at each level is drawn in Figure 6.2.

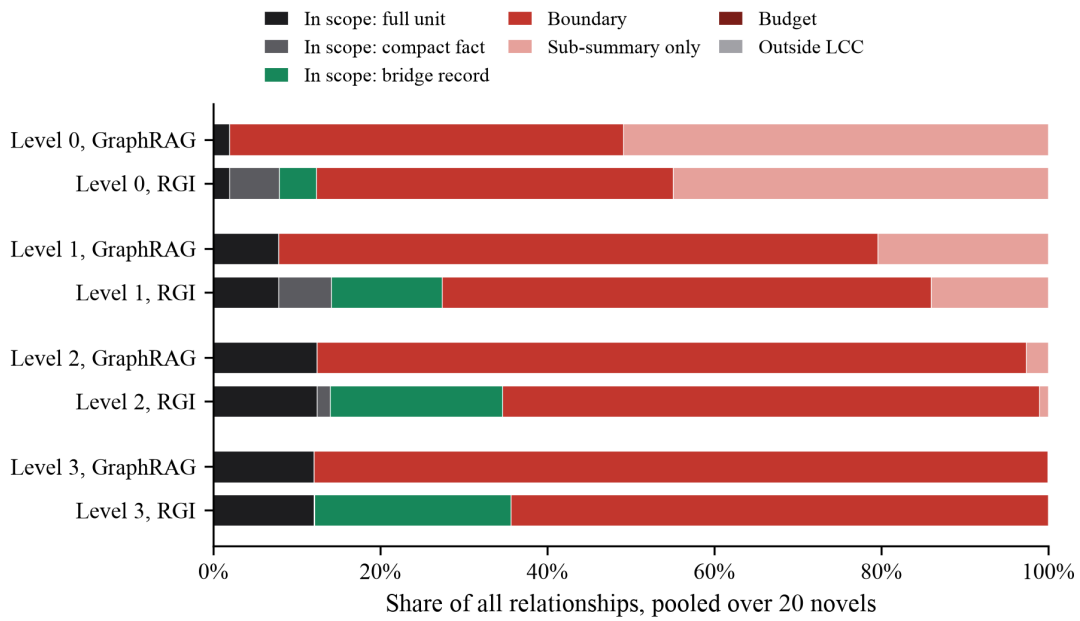


Figure 6.2: Where every relationship ends up at each query level

Table 6.3 shows how the summary inputs themselves were built. At the root, 98 of the 123 selected communities overflow and are summarized from their children; no leaf is cut, because every selected community at the root has children. At level 2, 93 leaf communities overflow and are cut at the budget, and 34 parents are assembled from child summaries.

Table 6.3: Summary inputs at B = 8,000, summed over all novels

Level	Fit	Trimmed	Sub-summary	Units (GraphRAG)	Units (RGI)	Bridge records	Compact facts
0	25	0	98	111	111	5,189	6,944
1	227	32	130	1,033	1,001	15,374	7,396
2	560	93	34	2,389	2,295	23,971	1,842
3	683	113	1	2,793	2,679	27,312	73

6.3 SCOPE AND EVIDENCE

Being out of scope is not the same as having no evidence in any input. Table 6.4 grades every relationship by the strongest way its evidence reaches a selected input, using the tiers of Table 4.2.

Table 6.4: Strongest form in which evidence reaches a selected input (%), B = 8,000

Level	Index	In scope	Endpoint	Elsewhere	Absent
0	GraphRAG	1.9	2.4	8.1	87.6
	RGI	12.4	2.0	7.1	78.5
1	GraphRAG	7.8	30.1	23.6	38.5
	RGI	27.4	23.0	15.8	33.8
2	GraphRAG	12.4	75.8	11.4	0.4
	RGI	34.6	56.3	8.7	0.3
3	GraphRAG	12.1	84.6	3.3	0.1
	RGI	35.6	61.3	3.0	0.1

In the fast pipeline an input is made of text units, and one unit names many entities. At level 2, 88.2% of relationships have a supporting unit in the input of a community that holds one of their entities, and only 0.4% have no evidence in any input at all. At level 0 the inputs of oversized communities contain no raw text, and 87.6% of relationships have no evidence anywhere. Proposition 1 applies to exactly these relationships. The fast pipeline therefore loses evidence mainly at coarse levels, through second-hand summarization, and loses scope mainly at fine levels, through boundaries.

6.4 SELECTION IN THE STANDARD PIPELINE

In the fast pipeline the input builder works with text units and lets every relationship through, but the standard pipeline's builder works with relationship records and filters them. We ran that builder, unchanged, on the census graphs, and Table 6.5 gives the outcome.

Table 6.5: How many within-community relationships reach the standard pipeline's summary input

Level	Communities	Within-community relationships	Reach the input	Share	Upper bound	Cross-community in inputs
0	123	61,415	2,841	4.6%	4,823	1,982
1	380	32,684	1,728	5.3%	4,768	3,047

Level	Communities	Within-community relationships	Reach the input	Share	Upper bound	Cross-community in inputs
2	504	14,319	897	6.3%	3,657	2,780
3	200	3,681	345	9.4%	1,154	832
4	19	370	53	14.3%	103	54
5	4	104	17	16.3%	23	7

Before the budget is even applied, only 4.6% to 16.3% of within-community relationships reach their community's candidate pool, at every level. Figure 6.3 sets the measured share against the upper bound from Proposition 3 of one relationship per entity. The measured share falls below the bound because the single record an entity contributes often points to an entity in some other community; summed over levels, the builder puts 8,702 cross-community relationships into inputs and leaves most of the internal ones out.

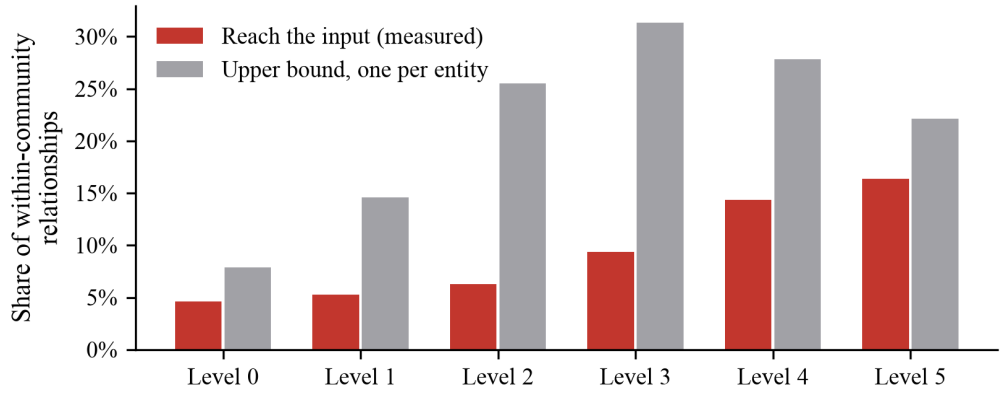


Figure 6.3: Measured share of within-community relationships in the standard summary input, set against the bound

With a pooled mean degree of 48.1, these graphs are dense, and the bound cuts deeply. A graph extracted by a language model is usually far sparser, and the same code would remove fewer of its relationships; the bound gives a practitioner an estimate of the effect from the mean internal degree alone. The behaviour exists in version 3.1.1, and the public issue report says it goes back to version 0.9.0 (Grenat, 2026). When we wrote this, a fix had been proposed but not merged.

6.5 WHAT RGI RECOVERS AT THE SAME BUDGET

Table 6.2 shows RGI raising the in-scope share at every level. At the root the share goes from 1.9% to 12.4%, and at level 2 from 12.4% to 34.6%, which takes 23,971 bridge records and 1,842 compact facts across the corpus. Averaged over the novels,

the level 2 gain is 24.6 percentage points, with a 95% confidence interval from 20.5 to 28.3. The gain for each novel is plotted in Figure 6.4. RGI improved 19 of the 20 novels, the exception being the smallest one, with 33 relationships, where both methods already keep the same share.

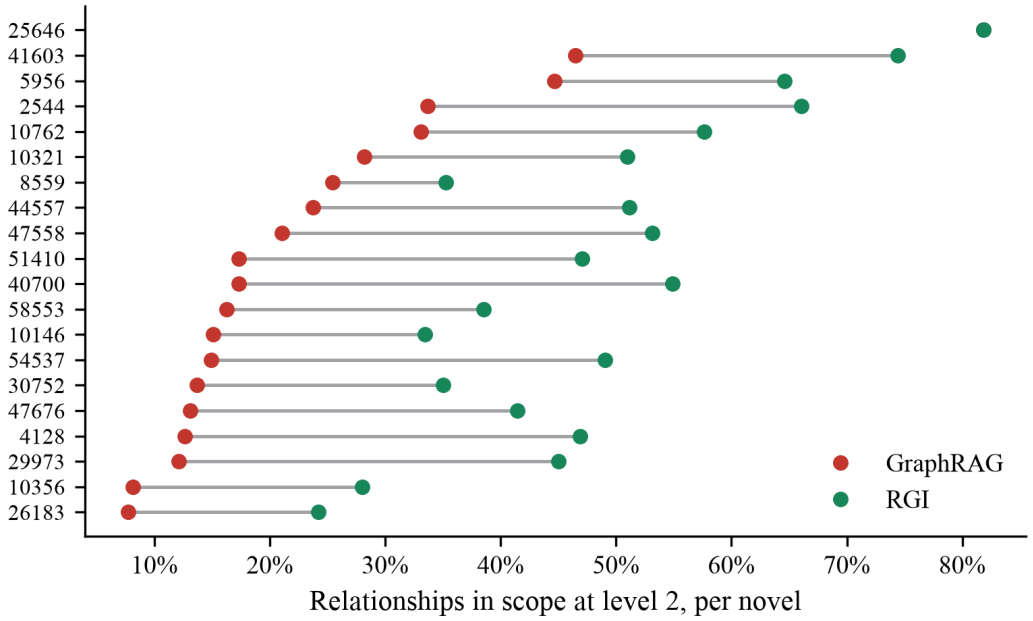


Figure 6.4: In-scope share at level 2 for each novel, GraphRAG and RGI

The guarantees held on every run. Across all 880 combinations of novel, level, budget and configuration, no relationship that GraphRAG kept in scope left scope under RGI (0 regressions), the fallback of the algorithm was never needed (0 times), no input exceeded 8,000 tokens at a budget of 8,000, and the coverage certificate held.

RGI also reduces absent evidence where Proposition 1 applies. At level 0 the share of relationships with no evidence in any input falls from 87.6% to 78.5%, and at level 1 from 38.5% to 33.8%. At level 2 it barely moves, from 0.4% to 0.3%, because there was little absent evidence to recover.

The two-tier layout does cost something, and the census lets us measure it. At level 2, RGI keeps 2,295 of the 2,389 text units that GraphRAG admits and uses the space of the rest for compact facts. Some relationships were visible only incidentally, in units that were replaced, and they lose that visibility. Summed over the four levels at a budget of 8,000 this happened in 570 cases, and at no level did it touch more than 0.21% of the relationships. None of these relationships went out of scope.

Figure 6.5 shows how the two methods react to the budget. At fine levels, a larger budget does less than RGI does: with 4,000 tokens RGI keeps 26.5% of level 2 relationships in scope, which is more than GraphRAG manages with 32,000 (14.9%). The root is different because its inputs overflow, and there the budget matters for both methods, with GraphRAG's share climbing from 0.2% to 27.7% over the same range and RGI's from 6.3% to 42.5%.

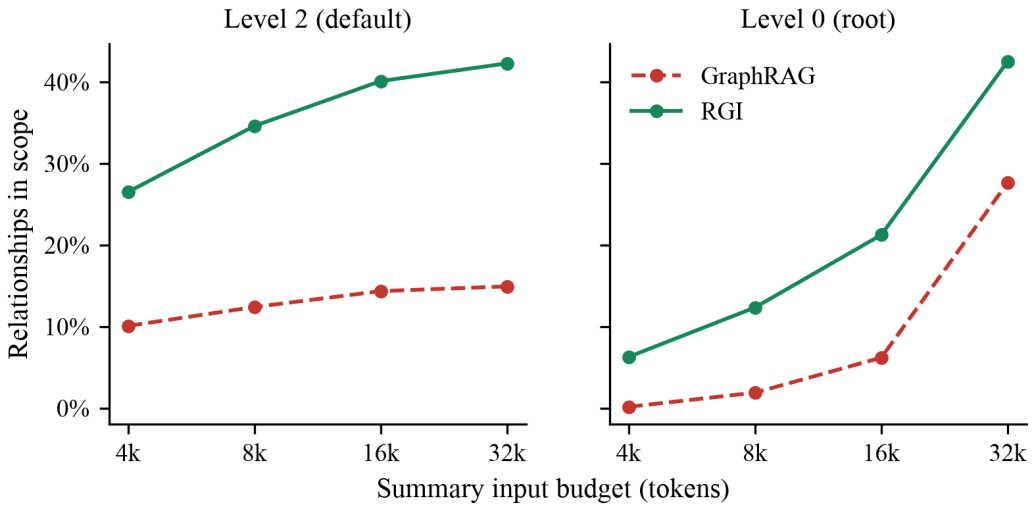


Figure 6.5: In-scope share against the summary input budget

6.6 IS THE LOST CONTENT WHAT QUESTIONS NEED?

How much of the gold evidence we could link to the graph is shown in Table 6.6. Of the 4,371 gold triples, 99.3% could be parsed and 45.8% linked at both endpoints, while 29.8% resolved to a relationship in the graph.

Table 6.6: Linking gold triples to the census graphs

Step	Triples	Share
Gold triples in the benchmark	4,371	100.0%
Parsed into (head, relation, tail)	4,339	99.3%
Both endpoints linked (token_set_ratio >= 85)	2,002	45.8%
Resolved to a relationship in the graph	1,302	29.8%

That puts the unlink rate at 54.2%, far above the 15% limit we froze in advance, and by our own rule this section cannot count as gate G2. What it does describe are the 840 questions (41.8% of the benchmark) whose gold evidence resolves. These questions are skewed toward prominent entities, which the extractor picks up more

reliably and which RGI's priority order also favours. The numbers below therefore describe this subset and should not be taken as estimates for the benchmark as a whole.

Keeping that in mind, Table 6.7 and Figure 6.6 show that at level 2, 79.9% of these questions have at least one gold relationship out of scope under GraphRAG and 15.2% under RGI. As Table 6.8 shows, the reduction appears at every level and for all four question types.

Table 6.7: Share of questions (%) with at least one gold relationship out of scope

Level	Questions	Missing a gold relationship (GraphRAG)	Same under RGI	Change [95% CI]	Gold relationships in scope (GraphRAG)	Same under RGI
Level 0	840	95.6	48.3	-47.3 [43.9, 50.6]	4.8	56.3
Level 1	840	88.1	23.8	-64.3 [61.0, 67.5]	13.9	79.6
Level 2	840	79.9	15.2	-64.6 [61.4, 67.9]	25.5	87.9
Level 3	840	79.4	14.9	-64.5 [61.2, 67.7]	26.3	88.2

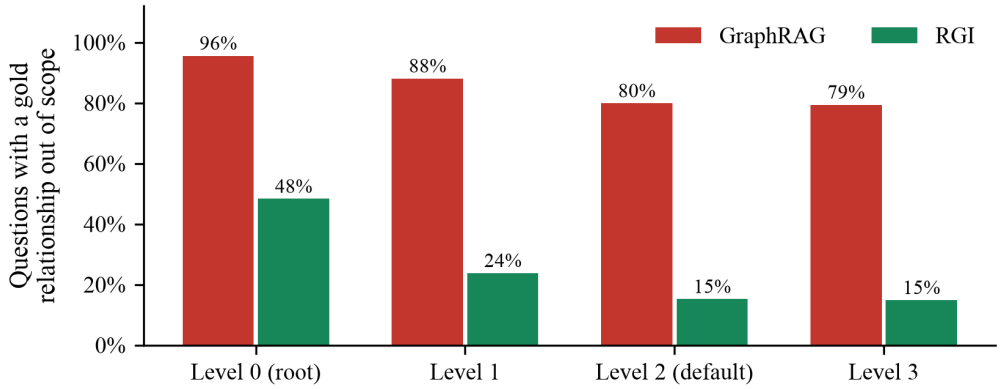


Figure 6.6: Share of linked questions with a gold relationship out of scope

Table 6.8: Questions with a gold relationship out of scope at level 2, by type (%)

Question type (level 2)	Questions	GraphRAG	RGI	Change
Complex Reasoning	317	84.5	17.0	-67.5
Contextual Summarize	183	79.8	13.7	-66.1
Creative Generation	52	98.1	15.4	-82.7
Fact Retrieval	288	71.5	14.2	-57.3

Evidence gives a different picture by level. At level 2, 99.8% of gold relationships have a supporting text unit in some input under GraphRAG, so their loss at that level is a loss of scope. At the root only 24.6% do, against 65.3% under RGI,

and the share of these questions with at least one gold relationship whose evidence is absent from every input falls from 79.8% to 40.7%.

6.7 SENSITIVITY AND ABLATIONS

How dense the graph is depends on the co-occurrence window, so Table 6.9 repeats the main configurations with text units of 600 and 300 tokens. With the shorter units the mean degree drops from 48.1 to 31.0 and 19.2. GraphRAG's in-scope share at level 2 goes up from 12.4% to 19.9% and 27.6%, and RGI's from 34.6% to 43.6% and 57.0%, with no regressions at any size. The amount of loss therefore changes with the graph, but at every unit size we tried the loss was there and RGI improved on GraphRAG.

Table 6.9: Sensitivity to the text-unit size (in-scope share, %, B = 8,000)

Text-unit size	Units	Relationships	Mean degree	Level 2 GraphRAG	Level 2 RGI	Level 0 GraphRAG	Level 0 RGI
1,200 (default)	1,024	116,217	48.1	12.4	34.6	1.9	12.4
600	2,239	94,161	31.0	19.9	43.6	6.1	19.8
300	5,584	89,273	19.2	27.6	57.0	26.1	41.8

In Table 6.10, one RGI parameter is varied at a time. Most inputs fit at level 2, and there the bridge records account for most of the gain; without them RGI reaches only 14.7%, through compact facts in the few inputs that overflow. The minimum support m has a bigger effect than the share b. Changing b from 0.06 to 0.24 moves the in-scope share from 29.4% to 37.2%, whereas changing m from 3 to 1 moves it from 21.9% to 49.8%.

Table 6.10: RGI ablations at B = 8,000 (% of relationships)

Configuration	Level 2 in scope	Level 2 bridged	Level 0 in scope	Level 0 bridged
GraphRAG (baseline)	12.4	n/a	1.9	n/a
No bridge records (b = 0)	14.7	0.0	13.4	0.0
Bridge share b = 0.06	29.4	15.0	13.2	2.6
Default (b = 0.12, f = 0.8, m = 2)	34.6	20.6	12.4	4.5
Bridge share b = 0.24	37.2	24.5	11.1	6.4
Full-item cutoff f = 0.6	35.5	20.6	21.5	4.5
Full-item cutoff f = 1.0 (no compact tier)	31.2	18.8	2.3	0.4
Minimum support m = 1	49.8	35.8	13.0	5.6

Configuration	Level 2 in scope	Level 2 bridged	Level 0 in scope	Level 0 bridged
Minimum support m = 3	21.9	7.7	12.2	2.8

The root behaves differently, because there bridges and compact facts compete for the same tail of the budget. Taking the bridges out raises the in-scope share (13.4% against 12.4%), a lower cutoff of f = 0.6 raises it further to 21.5% by leaving compact facts more room, and with no compact tier at all (f = 1.0) RGI keeps only 2.3%. How the budget is best divided thus depends on the level, which we come back to in Section 6.9.

6.8 CONTROLLED SIMULATION

The census ends at the summary input. To follow the loss through retrieval to judged answers we use the GraphRAG X-Ray simulation described in Section 4.7. Its results come from a model we wrote, so they illustrate the mechanism and are no substitute for measurements on real data. Results at each level are in Table 6.11 and Figure 6.7.

Table 6.11: Controlled simulation results (synthetic data, 48 questions, %)

Level (communities)	Retained, original	Retained, RGI	Evidence retrieved, original	Evidence retrieved, RGI	Judged correct, original	Judged correct, RGI
Leaf (22)	72.6	78.8	49.2	75.7	41.7	56.2
Community (8)	84.2	88.4	69.1	84.7	52.1	62.5
Higher (3)	46.3	65.2	80.2	87.0	60.4	66.7
Root (1)	14.4	23.4	54.3	71.2	43.8	50.0

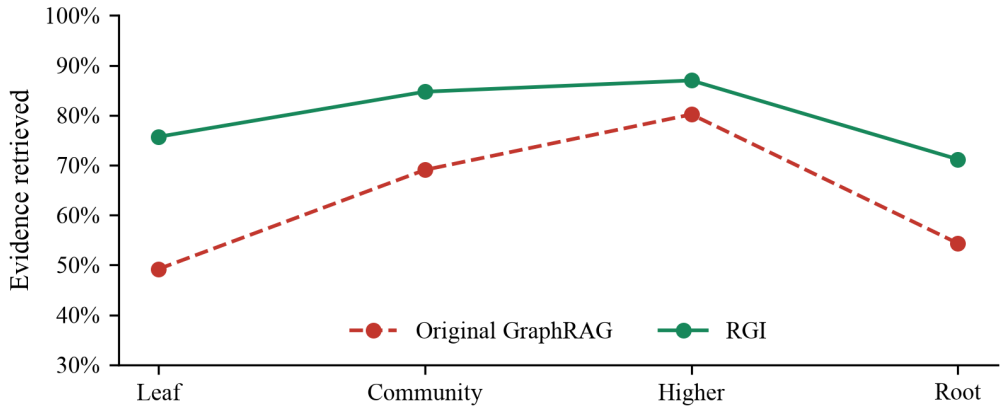


Figure 6.7: Simulated share of evidence retrieved at each level

At every level RGI retrieves more evidence. The paired difference in evidence retrieved at the leaf level lies between 18.0 and 35.9 percentage points (95% bootstrap interval over questions), and over all four levels no question and no relationship regressed (0 cases). Table 6.12 shows how the simulated results change with each parameter.

Table 6.12: Simulation results under each configuration (%)

Setting	Leaf evidence retrieved (original)	Leaf evidence retrieved (RGI)	Root relationships retained (RGI)
Reports read K = 2	48.5	74.5	23.4
Reports read K = 3	49.2	75.7	23.4
Reports read K = 6	49.2	76.2	23.4
Bridge share 0	49.2	49.2	23.4
Bridge share 0.06	49.2	75.7	23.4
Bridge share 0.12	49.2	75.7	23.4
Bridge share 0.24	49.2	75.7	23.4
Bridge threshold weight 3	49.2	89.6	23.4
Bridge threshold weight 4	49.2	75.7	23.4
Bridge threshold weight 5	49.2	61.4	23.4
Bridge threshold weight off	49.2	49.2	23.4
Full-item cutoff 0.6	49.2	75.7	33.7
Full-item cutoff 0.8	49.2	75.7	23.4
Full-item cutoff 1.0	49.2	75.7	14.5
Budget 4,000 tokens	49.2	75.7	11.6
Budget 8,000 tokens	49.2	75.7	23.4
Budget 16,000 tokens	49.2	75.7	48.8

We see three patterns in the sweeps. Reading more summaries does little for the original index: going from K = 2 to K = 6 raises leaf-level evidence retrieval only from 48.5% to 49.2%, since the lost relationships are not in any summary. The choice of which relationships to bridge also matters more than the budget given to bridges. Lowering the bridge threshold from weight 5 to weight 3 lifts retrieval from 61.4% to 89.6%, but raising the bridge share from 6% to 24% leaves it where it was (75.7% and 75.7%), the same ordering we saw on real data in Section 6.7. Finally, a larger budget helps both methods at the root, where the original keeps 6.3% of relationships with 4,000 tokens and 29.9% with 16,000. That gain is paid for in tokens on every query, while the residual adds none.

6.9 DISCUSSION

6.9.1 SCOPE AND EVIDENCE ARE DIFFERENT LOSSES

The census separates two failures that are easy to confuse. Boundaries take relationships out of scope at fine levels, but in the fast pipeline the text that supports them usually still reaches a summarizer whose report is about one of the entities involved. Whether that report then mentions the relationship is up to the model. Second-hand summarization at coarse levels removes the evidence itself, and there Proposition 1 applies in full. A study that reported only one of the two would misjudge the problem, in one direction or the other.

6.9.2 WHAT RGI DOES AND DOES NOT DO

RGI is an accounting layer. It makes boundary relationships explicit in the inputs where they matter, restores relationships that the budget would drop, and records everything else. It does not change the clustering, so it cannot remove multi-topic loss, and it cannot make a summarizer use what it is given. Its compact facts carry less information than the text they replace: in the fast pipeline a compact fact holds only the pair of entities and the number of supporting units. In the standard pipeline a bridge record can carry the relationship's extracted description.

6.9.3 ALLOCATION DEPENDS ON THE LEVEL

Most inputs fit at fine levels, so the slack in each input costs nothing and bridges are the only thing RGI can add. At the root, bridges and compact facts compete for the same tokens, and the ablations favour compact facts. Any fixed split of the budget is a compromise between these two situations, and a split that depends on the level, or a salience score learned from questions, would be a natural next step.

6.9.4 RECONCILING WITH EARLIER EVIDENCE

Edge et al. (2024a) found no significant difference between global search and a summarization baseline with no partition. That baseline has no boundary loss by construction, so the finding is evidence that partition loss does not move aggregate answer quality on broad sensemaking questions. Our claim is narrower and is measured element by element. We expect effects on fact retrieval and multi-hop questions whose evidence crosses communities, and the pre-registered protocol tests that by question type. LightRAG (Guo et al., 2024) avoids the community layer altogether and is

competitive on GraphRAG-Bench. Community summaries give a navigable view of a corpus that direct entity retrieval does not offer, and RGI tries to keep that view while accounting for what it costs.

6.10 THREATS TO VALIDITY

Several limits apply to these results. Table 6.13 lists them together with what was done about each.

Table 6.13: Threats to validity and how they were handled

Threat	Why it matters	What was done
Substitute extractor	Our proper-noun extractor omits the multi-word common noun phrases that GraphRAG's default extractor keeps, so absolute shares would differ	Kept the default's output format, stop words and token rules; every later stage is GraphRAG's own code; the selection analysis does not depend on extraction
Fast pipeline	Its inputs are text units; the standard pipeline's are model-written descriptions whose lengths only a model run produces	Measured the standard builder's selection exactly; left its budget loss to the model run
Graph density	Co-occurrence within 1,200-token units gives dense graphs (mean degree 48.1), which inflates boundary and selection loss	Repeated the census at 600 and 300 tokens
Gold linkage	More than half of the gold triples do not link, and those that do favour prominent entities	Reported the subset descriptively and did not treat it as gate G2
Scope versus content	Being in an input does not guarantee being in the summary, and being out of scope does not guarantee absence from it	Reported evidence tiers; gate G1 measures the second gap
Concentration	One novel holds 34% of all relationships	Reported per-novel means with intervals next to every pooled figure
Version	Results describe GraphRAG 3.1.1; a merged fix for selection would change Section 6.4	Recorded the version in every result file
Simulation	The simulated corpus tail and judge are generated	Labelled every simulated number as synthetic

Chapter 7

Conclusion and Future Work

7.1 CONCLUSION

With community-summary GraphRAG, what can be known about a corpus is settled before anyone asks a question. Our aim was to find where in the pipeline that happens and how much content is discarded there, and then to change the index so the discarded content is kept, or at least recorded, while GraphRAG's summaries, budget and search stay as they are.

We found five places where relationships leave the path to an answer: community boundaries, the input budget, the selection step of the standard input builder, the summarization of large communities from their children's summaries, and entities that are never clustered. Running GraphRAG 3.1.1's own indexing code on all twenty novels of GraphRAG-Bench Novel, with no language model, we measured each of them exactly. At the default query level the summary inputs keep 12.4% of 116,217 relationships in scope. At the root, 87.6% of relationships have no evidence in any input at all. The standard input builder passes only 4.6% to 16.3% of within-community relationships into its inputs, before the budget is even applied.

Residual Graph Indexing keeps GraphRAG's summaries and search as they are, spends the same budget differently, and keeps an account of every relationship. At the default level it raises the in-scope share from 12.4% to 34.6%, a mean gain of 24.6 points per novel with a 95% confidence interval of 20.5 to 28.3. It never dropped a relationship that GraphRAG kept, never exceeded the budget and never needed its fallback, across 880 combinations of novel, level, budget and configuration. Whether that changes answers is the question the pre-registered protocol was built to settle, and it remains open.

7.2 OBJECTIVES REVISITED

Table 7.1 compares the outcome with each objective set in Section 1.5. Four objectives were met in full. Two were met in part, and in both cases the missing part is the language model run.

Table 7.1: Objectives and outcomes

Objective	Status	Evidence
Model of index-time loss with scope and evidence separated	Met	Loss model, evidence tiers and three propositions (Section 4.3, Section 4.4)
Tested toolkit with thresholds frozen before data	Met	Nine modules, ten scripts, 166 passing tests; thresholds committed on 15 August 2026
Exact census on GraphRAG-Bench Novel with GraphRAG's own code	Met, with one substitution	All 20 novels, four levels, four budgets; substitute extractor disclosed
RGI with stated guarantees, checked on every run	Met	Zero regressions, zero fallbacks, certificate held on every run
Estimate how much lost content questions need	Partly met	840 linked questions analysed; unlink rate 54.2% rules out gate G2
Simulator showing effects on retrieval and answers	Met for the simulation	GraphRAG X-Ray with typed questions; answer effects are synthetic

7.3 WHAT THE PROJECT SHOWED

Some findings were not obvious when the work began.

The first is that boundary loss and evidence loss are different problems. We expected the fine levels of the hierarchy to be where content disappears, because that is where the boundaries are. In the fast pipeline, though, the text that supports a boundary relationship almost always still reaches some summarizer at those levels. The real absence of evidence is at the coarse levels, where large communities are written from their children's summaries. A claim that content is unreachable has to be made about the second problem, not the first.

The second is the selection behaviour of the standard input builder. We set out to measure the effect of the budget, and found that the builder removes most within-community relationships before the budget plays any part, because it keeps one relationship per entity. The effect depends strongly on graph density, and Proposition 3 gives a simple way to estimate it.

A third finding was about where effort is best spent. In the census and in the simulation alike, the choice of which relationships to bridge made far more difference than the amount of budget the bridges received. At fine levels, RGI working with half the default budget kept more relationships in scope than GraphRAG did with four times

the default, so putting content explicitly into the right inputs did more good than simply giving the inputs more room.

The fourth finding concerns benchmarks. In GraphRAG-Bench the gold evidence sentences restate the source text instead of quoting it, and more than half of the gold triples cannot be linked to an automatically extracted graph. We discovered this before running any gate, because the linking rule and its failure threshold had been written down beforehand. If we had tuned the linking after looking at the results, the gold analysis would have looked stronger, but part of that strength would have come from the tuning.

7.4 LIMITATIONS

The biggest limitation is that answer accuracy was never measured on a real index built by a language model. The team's API key failed validation, and a free tier could not have handled the number of calls the study needs. Every part of the work that does not depend on the model was completed and the rest was prepared, but the central claim, that the lost content changes answers, is still untested on real data.

The census has three further limits. We could not install GraphRAG's default noun-phrase extractor and used a substitute that keeps its output format without reproducing all of its behaviour. The graphs from the fast pipeline are also much denser than graphs extracted by a language model usually are, which makes boundary and selection loss larger, although the sensitivity runs show RGI's gain holding as density falls. Finally, the gold-evidence analysis covers only a biased subset of the questions.

Two parts of the planned toolkit have not been verified yet. The truncation replay needs a written reading of GraphRAG's summarization code, which was started but is unfinished, and the NLI model chosen for gate G1 has not yet been loaded under the installed version of the transformers library. The simulator's effects on answers, as noted before, come from a judge we wrote and should be read as an illustration of the mechanism.

7.5 FUTURE WORK

Most of what remains to be done comes straight from the limits listed above:

1. Build an index of GraphRAG-Bench Novel with a language model and run the four gates in their fixed order, with the two annotators for G1 and G3. This settles whether index-time loss changes answers, by question type.
2. Compare GraphRAG, RGI, RGI with placebo residuals and LightRAG at equal token counts, and replicate the study on the Medical corpus of GraphRAG-Bench.
3. Implement query-time residual retrieval in full and measure what the stored facts add beyond the summaries, with the token-exact swap already in the toolkit.
4. Replace the fixed bridge rule with an allocation that depends on the level, or with a salience score learned from past questions.
5. Add short pointers that let multi-topic entities appear in the communities they were not filed under, which is the one form of loss RGI does not address.
6. Repeat the census with GraphRAG's default extractor once its data can be installed, and with a sparser graph extracted by a language model.
7. Share the selection measurements with the GraphRAG maintainers on the open issue, and prepare the research paper for publication, first as a preprint and then at a venue once the answer-level results are in.

7.6 BENEFITS OF THE WORK

The work is useful even before the answer-level test is run. Anyone who builds a GraphRAG index can run the audit and learn, for their own corpus and configuration, how much of the extracted graph their summaries can never see, and through which mechanism. The coverage certificate turns an invisible loss into a recorded one. RGI costs no extra model calls and no extra tokens, so it can be adopted without changing the economics of an index. The census, the toolkit and the simulator are all reproducible from public data on an ordinary laptop, which makes the findings easy to check and to extend.

REFERENCES

[1]. Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment, 2008*(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

[2]. Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., & Yu, D. (2024). Dense X retrieval: What retrieval granularity should we use? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 15159-15177). Association for Computational Linguistics.

[3]. Chung, J., Pedigo, B. D., Bridgeford, E. W., Varjavand, B. K., Helm, H. S., & Vogelstein, J. T. (2019). GraSPy: Graph statistics in Python. *Journal of Machine Learning Research, 20*(158), 1-7.

[4]. Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37-46. https://doi.org/10.1177/001316446002000104

[5]. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., & Larson, J. (2024a). *From local to global: A graph RAG approach to query-focused summarization* (arXiv:2404.16130). arXiv. https://doi.org/10.48550/arXiv.2404.16130

[6]. Edge, D., Trinh, H., & Larson, J. (2024b, November 25). *LazyGraphRAG: Setting a new standard for quality and cost*. Microsoft Research Blog. https://www.microsoft.com/en-us/research/blog/lazygraphrag-setting-a-new-standard-for-quality-and-cost/

[7]. Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics, 7*(1), 1-26. https://doi.org/10.1214/aos/1176344552

[8]. Fortunato, S. (2010). Community detection in graphs. *Physics Reports, 486*(3-5), 75-174. https://doi.org/10.1016/j.physrep.2009.11.002

[9]. Fortunato, S., & Barthélemy, M. (2007). Resolution limit in community detection. *Proceedings of the National Academy of Sciences, 104*(1), 36-41. https://doi.org/10.1073/pnas.0605965104

[10]. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). *Retrieval-augmented generation for large language models: A survey* (arXiv:2312.10997). arXiv. https://doi.org/10.48550/arXiv.2312.10997

[11]. Good, B. H., de Montjoye, Y.-A., & Clauset, A. (2010). Performance of modularity maximization in practical contexts. *Physical Review E, 81*(4), 046106. https://doi.org/10.1103/PhysRevE.81.046106

[12]. Grenat, E. (2026, September 30). *[Bug]: Community report context seems to keep only one edge per node* (Issue No. 2583) [GitHub issue]. microsoft/graphrag. https://github.com/microsoft/graphrag/issues/2583

[13]. Guimerà, R., & Amaral, L. A. N. (2005). Functional cartography of complex metabolic networks. *Nature, 433*(7028), 895-900. https://doi.org/10.1038/nature03288

[14]. Guo, Z., Xia, L., Yu, Y., Ao, T., & Huang, C. (2024). *LightRAG: Simple and fast retrieval-augmented generation* (arXiv:2410.05779). arXiv. https://doi.org/10.48550/arXiv.2410.05779

[15]. Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems 37*.

[16]. Han, H., Ma, L., Wang, Y., Shomer, H., Lei, Y., Qi, Z., Guo, K., Hua, Z., Long, B., Liu, H., Aggarwal, C. C., & Tang, J. (2025). *RAG vs. GraphRAG: A systematic evaluation and key insights* (arXiv:2502.11371). arXiv. https://doi.org/10.48550/arXiv.2502.11371

[17]. He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*.

[18]. He, X., Tian, Y., Sun, Y., Chawla, N. V., Laurent, T., LeCun, Y., Bresson, X., & Hooi, B. (2024). G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems 37*.

[19]. Hossain, J., & Sarıyüce, A. E. (2026). *Core-based hierarchies for efficient GraphRAG* (arXiv:2603.05207). arXiv. https://doi.org/10.48550/arXiv.2603.05207

[20]. Huang, H., Huang, Y., Yang, J., Pan, Z., Chen, Y., Ma, K., Chen, H., & Cheng, J. (2025). Retrieval-augmented generation with hierarchical knowledge. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics.

[21]. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459-9474).

[22]. Li, Y., Yang, K., Wang, T., Chen, B., Li, B., & Mao, C. (2026). *Deep GraphRAG: A balanced approach to hierarchical retrieval and adaptive integration* (arXiv:2601.11144). arXiv. https://doi.org/10.48550/arXiv.2601.11144

[23]. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157-173. https://doi.org/10.1162/tacl_a_00638

[24]. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153-157. https://doi.org/10.1007/BF02295996

[25]. Microsoft. (2026). *GraphRAG* (Version 3.1.1) [Computer software]. https://github.com/microsoft/graphrag

[26]. Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences, 115*(11), 2600-2606. https://doi.org/10.1073/pnas.1708274114

[27]. Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., & Tang, S. (2024). *Graph retrieval-augmented generation: A survey* (arXiv:2408.08921). arXiv. https://doi.org/10.48550/arXiv.2408.08921

[28]. Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*.

[29]. Tang, Y., & Yang, Y. (2024). *MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries* (arXiv:2401.15391). arXiv. https://doi.org/10.48550/arXiv.2401.15391

[30]. Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports, 9*, 5233. https://doi.org/10.1038/s41598-019-41695-z

[31]. Wang, J., Huang, H., Ge, X., Su, J., Liu, W., & Lian, S. (2026). *OMD-GraphRAG: Enhancing GraphRAG with ontology-guided extraction, multi-dimensional clustering and dual-channel fusion* (arXiv:2603.25152). arXiv. https://doi.org/10.48550/arXiv.2603.25152

[32]. Wang, S., Fang, Y., Zhou, Y., Liu, X., & Ma, Y. (2026). ArchRAG: Attributed community-based hierarchical retrieval-augmented generation. *Proceedings of the AAAI Conference on Artificial Intelligence, 40*(19), 15868-15876. https://doi.org/10.1609/aaai.v40i19.38619

[33]. Wiegand, T., Sullivan, G. J., Bjøntegaard, G., & Luthra, A. (2003). Overview of the H.264/AVC video coding standard. *IEEE Transactions on Circuits and*

Systems for Video Technology, 13(7), 560-576. https://doi.org/10.1109/TCSVT.2003.815165

[34]. Xiang, Z., Wu, C., Zhang, Q., Chen, S., Hong, Z., Huang, X., & Su, J. (2026). When to use graphs in RAG: A comprehensive analysis for graph retrieval-augmented generation. In *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2506.05690

[35]. Xu, F., Shi, W., & Choi, E. (2024). RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation. In *International Conference on Learning Representations*.

[36]. Zhang, Y., Wu, R., Cai, P., Wang, X., Yan, G., Mao, S., Wang, D., & Shi, B. (2026). LeanRAG: Knowledge-graph-based generation with semantic aggregation and hierarchical retrieval. *Proceedings of the AAAI Conference on Artificial Intelligence, 40*(41). https://doi.org/10.1609/aaai.v40i41.40789

[37]. Zhou, Y., Su, Y., Sun, Y., Wang, S., Wang, T., He, R., Zhang, Y., Liang, S., Liu, X., Ma, Y., & Fang, Y. (2025). In-depth analysis of graph-based RAG in a unified framework. *Proceedings of the VLDB Endowment, 18*(13), 5623-5637.

APPENDICES

APPENDIX 1: PRE-REGISTERED KILL CRITERIA

The definitions in this appendix are the ones committed to the project repository on 15 August 2026, before any data were collected, and the thresholds themselves are in Table 3.5. Writing the definitions down in advance leaves no room to argue, once the data are in, that a gate which fired did not really fire. Gates G1 to G3 fire on their point estimates and G4 on its confidence interval. When a gate fires the study stops at that point: the claim it protects is dropped, the later experiments are not run, and the result is reported as a null.

Gate G1 uses 300 relationships drawn from the permanent cross edges, those that no level contains, stratified by relationship weight into four quartiles of 75 each. For each one, the premise is the passages about its two entities taken from the reports of the two communities involved, and the hypothesis is the relationship's own description. The metric is the share classified as entailed by the NLI model, reported overall and per quartile with bootstrap intervals. The NLI model may not be used until two human annotators have labelled 100 items and reached a Cohen's kappa of at least 0.7.

Gate G2 measures the share of benchmark questions whose gold set intersects the loss set. The loss set for this gate includes boundary and budget loss only. Multi-topic loss is excluded by construction, because a misassigned entity is still read by global search, in the wrong framing. The gate is reported as inconclusive if more than 15% of gold items fail to link.

Gate G3 measures the share of failed answers assigned to the category "index-absent", where a failure that fits several categories is assigned the earliest one in the pipeline. Two annotators label the first 50 failures independently; below a kappa of 0.7 the automated classifier is not used.

Gate G4 is the paired difference in accuracy between the oracle and placebo arms over the same questions, with a 95% bootstrap interval from at least 1,000 resamples by question, reported with McNemar's test and Cohen's h. Its preconditions run in a fixed order: the injection must be token-exact on 50 random examples, then the placebo arm runs, then the oracle arm.

In the same commit we also fixed what would count as a strong result, so that the meaning of "strong" could not be changed after the numbers came in. These markers, listed in Table A1.1, do not stop the study. They only set how strongly its findings may be stated.

Table A1.1: Strength markers committed with the gates

Measurement	Weak	Strong
G1 recoverability on high-weight edges	n/a	below 40%
G2 share of questions needing lost content	n/a	above 25%
G3 share of index-absent failures	n/a	the largest category
G4 oracle minus placebo	n/a	above 5 points
E1 weighted share of cross edges at the operating level	above 8%	above 25%
E4 leaf communities overflowing at 8,000 tokens	above 20%	still above at 16,000
E3 multi-topic rate among entities in gold evidence	above 15%	plus a clear partial correlation with degree

Later that day a second commit froze the linking parameters used by gate G2, along with the tokenizer used throughout the study; both are given in Table A1.2.

Table A1.2: Frozen linking parameters

Parameter	Frozen value
Strategy	Link gold triples (the sentence strategy is kept but not run)
Scorer	RapidFuzz `token_set_ratio`
Threshold	At least 85, for both head and tail
Matched against	Entity titles
Normalization	Lowercase, punctuation removed, whitespace collapsed
Candidate entities	Restricted to the question's own novel
Tokenizer for all token counts	tiktoken `o200k_base`

APPENDIX 2: RGI SUMMARY-INPUT CONSTRUCTION

Below is the part of the census script that builds a single RGI summary input, following the algorithm in Section 4.5. To fit the page we wrapped long lines and shortened a few helper calls, without changing the logic.

```
# GraphRAG's own input for this community: mode, admitted units, tokens
mode, k_base, base_tokens = base[cid]
facts = []
if mode == "fits":
    k, used = k_base, base_tokens    # keep everything; slack goes to bridges
    pending = []
elif mode == "trimmed":
    k = (ctx.admit(key, recs, int(budget * rgi.full_cut))
         if rgi.full_cut < 1 else k_base)
    k = min(k, k_base)
    used = ctx.tokens(key, recs, k)
    have = covered_by(cid, k)
    base_cov = covered_by(cid, k_base)
    # monotonicity: what GraphRAG's own prefix represented is written first
    reserved = sorted(base_cov - have, key=prio.get)
    rest = sorted(set(c["intra"]) - base_cov, key=prio.get)
    cost = (tok.n(FACTS_HDR) if reserved else 0) + sum(
        tok.n(line(r)) + 1 for r in reserved)
    if used + cost > budget:                 # fallback to GraphRAG's own prefix
        fallback += 1
        k, used, reserved = k_base, base_tokens, []
    facts.extend(reserved)
    pending = rest
else:  # input built from child summaries: they get f*B, the tail is ours
    k, used = 0, int(budget * rgi.full_cut)
    pending = sorted(c["intra"], key=prio.get)

# bridge records: boundary relationships of this community, by priority
bridges, spent = [], 0
allowance = min(budget * rgi.bridge_share, budget - used_after_facts)
for r in sorted((r for r in boundary_of[cid]
                 if n_units(r) >= rgi.bridge_min_units), key=prio.get):
    cost = tok.n(line(r)) + 1 + (0 if bridges else tok.n(BRIDGE_HDR))
    if spent + cost > allowance:
        break
    bridges.append(r)
    spent += cost

# compact facts for in-scope relationships still missing, up to the budget
for r in pending:
    cost = tok.n(line(r)) + 1
    if used_after_bridges + cost > budget:
        break
    facts.append(r)
    used_after_bridges += cost

# exact check on the rendered text; drop trailing facts if merges overflow
total = tok.n(render(k, bridges, facts)) + (int(budget * rgi.full_cut)
                                            if mode == "second_order" else 0)
while total > budget and facts:
    facts.pop()
    total = tok.n(render(k, bridges, facts)) + ...
assert total <= budget
```

After every community has been built, the script assigns each relationship to the represented, bridged or stored set and checks the coverage certificate:

```
residual = {rid for rid, s in rstatus.items() if s != "covered"}
certificate = (set(how) | residual) == set(all_relationship_ids) \
              and not (set(how) & residual)
regressions = sum(1 for rid, s in status.items()
                  if s == "covered" and rstatus[rid] != "covered")
assert certificate
```

APPENDIX 3: CENSUS OUTPUT FILES

Each census run writes four tables. They are described in Table A3.1, and the columns of the main table are listed in Table A3.2.

Table A3.1: Files written by the census

File	One row per
`graph_stats.csv`	Novel: text units, tokens, extracted and pruned entities and relationships, communities per level, removed most-connected entity
`level_census.csv`	Novel, level, budget and method: counts of every status, tier and RGI component, plus checks
`gold_triples.csv`	Gold triple: whether it parsed and linked, its score and the number of relationships it resolved to
`gold_questions.csv`	Question and level: gold relationships in scope under GraphRAG and RGI, by status and tier

Table A3.2: Main columns of the level census

Column	Meaning
`covered`, `boundary`, `budget_lost`, `indirect`, `outside`	Relationships with each status (indirect means sub-summary)
`via_bridge`, `via_compact`	Relationships brought into scope by bridge records and by compact facts
`fits`, `trimmed`, `second_order`	Selected communities whose input fits, is cut, or is built from child summaries
`passages`, `facts`, `bridges`	Text units, compact facts and bridge records placed in inputs
`expo_scope`, `expo_endpoint`, `expo_elsewhere`, `expo_absent`	Relationships in each evidence tier
`max_tokens`, `mean_tokens`	Largest and mean input size in tokens
`regressions`, `fallback`, `certificate`	Checks on RGI's guarantees

APPENDIX 4: GRAPHRAG X-RAY USER GUIDE

Nothing has to be installed to use the simulator. It is enough to open `demo/index.html` in a modern browser, or to serve the folder locally with `python3 -m http.server` and open the address it prints. The controls are summarized in Table A4.1.

Table A4.1: Controls of GraphRAG X-Ray

Control	Effect
Question menu	Choose one of the five featured questions, or type a question about the novel
Method panel, or keys 1 and 2	Show the graph under original GraphRAG or under RGI
Level stepper, or keys [and]	Move between the leaf, community, higher community and root levels
Click a relationship, entity or community	Open an inspector that explains why it is kept or lost at this level
Zoom buttons, or Ctrl and scroll	Zoom and pan the graph
Theme button	Switch between light and dark themes
Escape	Close the menu or the open inspector

The question menu, with its five featured questions and a field for typing new ones, appears in Figure A4.1. Names in a typed question are matched to entities, and if none of them matches, the menu reports this and stays on the current question.

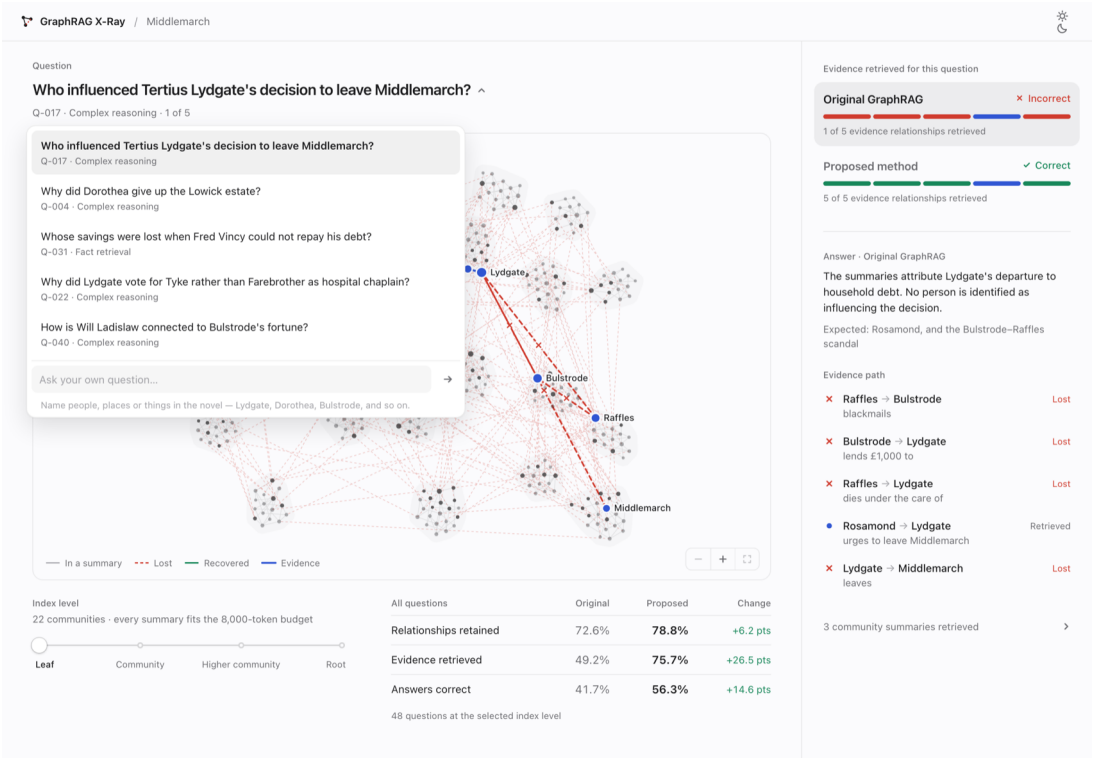


Figure A4.1: Question menu with featured questions and a field for typed questions

APPENDIX 5: REPRODUCTION GUIDE AND PROJECT TIMELINE

An ordinary laptop is enough to regenerate every result, in five steps:

1. Set up the Python 3.11 environment from the project's lock file, and install Node.js for the simulator scripts.
2. Using the commands in Table 5.4, run the census, the two sensitivity runs and the selection analysis. On the first run the `o200k_base` encoding is downloaded once and cached.
3. Run the simulation sweep under Node.js.
4. Run the analysis script. It writes every figure, table and number that appears in the paper and in this report.
5. Run `python -m pytest tests/` and check that all 166 of the toolkit's tests pass.

The main stages of the project are listed in Table A5.1. Most dates are taken from the repository's commit history; for files that have not been committed yet, we used their modification dates.

Table A5.1: Project timeline

Date	Work recorded
15 August 2026	Repository set up; kill thresholds frozen (commit d223de8); linking parameters and tokenizer frozen (commit 21e43d8)
15 August 2026	Measurement toolkit, synthetic fixture, 166 tests and the ten experiment scripts; handover document listing the seven differences from the documentation
15 to 16 September 2026	GraphRAG X-Ray simulator built and redesigned; design documents for RGI
13 October 2026	Census of all 20 novels, selection analysis, sensitivity runs and ablations; typed questions and theme switching in the simulator; research paper
14 to 16 October 2026	Final report